VigilODE 통합 연구보고서

실행 환경 복구에서 병렬 stage·구조 축약·하이브리드 AD·연속 오차 인증까지

이 대화 스레드의 이론·구현·벤치마크·복구 기록 통합

연구 기록: 2026 년 10 월 19–20 일
통합 편집: 2026 년 10 월 22 일

첨부 원문과 고정 커밋을 기준으로 작성했다. 새로운 solver 실행이나
성능 측정이 아니라, 기록된 결과·수정된 해석·남은 범위를 통합한 보고서다.

차 례

초록 2

1. 보고서의 범위와 근거 체계 2

2. 연구의 연속성과 단계별 산출물 3

3. 측정·정확도·재현성을 비교하는 공통 언어 4

4. 초기 실행 환경 복구와 첫 벤치마크 5

5. 정밀 재벤치마크: 수치 동일성, CT 2,700 조건, 계측 비용 7

6. 외부 비교의 정정과 엄격한 native 3 자 비교: PR71 9

7. Banded 정확성 수정과 exact LU 재사용: PR73 11

8. 통합 이론: matrix-free·동시 stage·homotopy 12

9. JVP-only coupled step 의 Rust 구현: PR74 14

10. Residual-first 와 후보 예산의 경제성: PR75 16

11. 최신 경로의 동일 호스트 중간점검: PR76 17

12. JVP 를 사용하지 않는 물리·대수적 경로의 발산 탐색 18

13. 연속 defect 로 root·시간·출력 오차를 연결하다 21

14. 격자·AMR·ALE 조언에 대한 구성적 교정 22

15. Exact 구조 축약·국소 AD·연속 trajectory 의 결합: PR77 23

16. 성과의 통합 해석과 미해결 영역 27

17. 출판·복구·백업과 결과의 재현성 28

18. 최종 결론 29

부록 A. 고정 소스·실행·게시 장부 29

부록 B. 시험·실패·정정의 통합 장부 31

부록 C. 재현 절차와 원자료 탐색 32

부록 D. 출처 등록부와 인용 범위 33

부록 E. 이번 통합 집필에서 수행한 검증 35

초록

본 보고서는 VigilODE 에 대해 이 대화 스레드에서 수행한 실행 환경 복구, 초기 성능·정확도 조사, 외부 ODE 솔버와의 비교, banded 연산의 결함 수정, matrix-free coupled-stage homotopy 의 구성과 Rust 구현, residual-first 비용 절감, 미분 없는 물리·대수적 축약, 하이브리드 국소 자동미분 (AD), adaptive trajectory 와 연속 출력 오차 인증, 그리고 결과의 원격 보존까지를 하나의 연구 흐름으로 통합한다. 사용자가 첨부한 여섯 문서와 이 스레드의 원래 보고서, 고정 커밋의 GitHub 기록을 일차 자료로 삼았다. 새로운 솔버 벤치마크를 수행한 보고서가 아니며, 과거의 실측과 이번 집필 과정의 제한된 산술·기호 검산을 분리한다.

전체 연구에서 확인된 개선은 서로 다른 세 층위에 놓인다. 첫째, 같은 적분법과 출력 결과를 유지하면서 workspace·저장 구조·불필요한 연산을 줄이는 구현 개선이다. 초기 400 변수 Brusselator 에서는 generic direct 와 banded-slices 사이에 약 53–57 배의 시간 차이가 관측됐지만, 이것은 버전 간 가속률이 아니라 같은 버전 안의 실행 경로 차이였다. 둘째, 같은 Rosenbrock stage target 을 공유 JVP 부분공간과 동시 보정으로 풀어 큰 공간의 여덟 순차 solve 를 대체하는 실행 알고리즘이다. 12,288 변수 합성 문제에서 작은 24 차원 stage-root 시스템과 실제 병렬 실행을 구현했으나, 그 병렬 경로가 기존 순차 솔버보다 항상 빠르지는 않았다. 셋째, 모델의 정확한 대수적 구조를 이용해 비선형 계산 자체를 줄이는 접근이다. 정확한 collective closure 가 있을 때의 packet 과, closure 가 깨졌을 때의 국소 AD 및 band-plus-rank-one solve 를 연속 defect 인증과 결합했다.

외부 비교는 초기의 해석을 수정하는 계기가 됐다. SciPy 1.17 의 C 이식 VODE wrapper 에서 발생한 analytical dense Jacobian 전치 문제를 원본 Fortran DVODE 의 열세로 해석한 주장은 철회됐다. 원본 Netlib DVODE 를 사용하고 실제 달성 오차를 맞춘 비교에서는 작은 stiff 문제에 VigilODE fast 의 강점이 있었지만, 공간 이산화 Brusselator 에서는 DVODE banded 가 VigilODE banded 보다 약 1.77–2.19 배 짧았다. 하이브리드 인증 trajectory 에서도 내부 후보 교체로 큰 개선을 얻었으나, 인증을 수행하지 않는 외부 BDF 보다 빠르다는 결과는 얻지 못했다.

정확도에 관한 가장 중요한 진전은 실행 성공, stage-root 수렴, embedded 시간오차 통과, 요청 격자의 경험적 오차, 연속 오차 인증을 서로 다른 판단으로 분리한 것이다. 초기 조밀 출력에서는 estimator 를 통과해도 실제 정규화 오차가 1 을 넘었다. 이후 명시적 scalar 모델에서는 미분 oracle 없이 상속 반경과 반환 반올림까지 포함한 adaptive 인증을 실행했고, 하이브리드 Rust 구현에서는 특정 생성형 반사 격자 ODE 의 실제 반환 곡선에 대한 연속 defect 와 causal 반경 전파를 연결했다. 이 성과는 범용 PDE·AMR·ALE 인증이나 전역 최적 자동 선택의 완성을 뜻하지 않는다.

최종적으로 이 연구가 지지하는 설계는 **단일한 주전 솔버의 강제 선택이 아니라, 모델의 구조와 계산 의무가 명시된 후보들을 동일한 물리적 출력 계약 아래 선택하는 구조**다. 성능의 목적함수는 미분 횟수나 root latency 하나가 아니라, 준비·검증·실패·fallback·remap 을 모두 포함한 승인된 시간 전진량당 비용이어야 한다. [A1–A6; R71; R73–R78]

1. 보고서의 범위와 근거 체계

1.1 연구 대상과 편집 경계

연구 대상은 `cosmosapjw-quantum/vigilode` 및 이 대화에서 생성한 독립 Python 탐색 자료다. 초기 Rust 설치와 두 차례 벤치마크부터 PR #71, #73–#78 에 이르는 기록을 포함한다. 다른 우주론·원자반응 저장소의 실행을 VigilODE 의 성능 근거로 가져오지 않았다. PR 번호가 연속되지 않는 것은 실제로 이 스레드에서 다룬 연구 가지를 따른 결과이며, PR #72 에 별도의 결과를 추정해 채우지 않았다.

첨부 문서의 과거형 상태는 작성 당시의 사실로 보존한다. 예컨대 하이브리드 복구 보고서의 “추가 push 미수행” 은 당시의 게시 제한을 뜻한다. 이후 PR #77 과 #78 에 해당 자료가 실제로 게시된 상태는 별도의 후속 사건으로 정리한다. 반대로 후속 게시가 과거 수치 실행의 성공 여부나 미완료 workspace 시험을 소급 변경하지는 않는다. [A5, §§1, 7; R78]

이 문서는 원자료를 모두 다시 적분한 독립 재현 연구가 아니다. 첨부 전체 문서를 읽고, 누락된 통합 연구와 고정 커밋의 핵심 보고서를 추가 확인하고, 표의 산술·근거 상태·대표 항등식을 교차 점검한 통합 저술이다. 도구가 없었던 독립인원 심사나 native subagent 실행을 수행했다고 표시하지 않는다.

1.2 근거 상태의 구분

표기	본 보고서에서의 의미
유도됨	전제와 계산으로 얻은 수학적 관계. 적용 domain 을 별도로 명시한다.
수치 검산됨	기록된 입력에서 참조·잔차·불변량·수렴 대조를 실행한 결과다. 일반 정리와 같지 않다.
구현 검증됨	명시한 코드·시험 대상에서 실제 실행 기록이 존재한다. 전체 저장소 검증으로 확대하지 않는다.
문헌·공식 자료로 지지됨	공식 소스나 원 논문이 그 좁은 주장을 뒷받침한다. 프로젝트의 실측을 대신하지 않는다.
제안·추론	기존 결과에서 도출한 설계 후보 또는 해석. 실행·성능·인증 완료와 구분한다.
미해결·미수행·차단	각각 과학적 불확실성, 실행하지 않은 범위, 환경·권한 등의 실행 장애를 뜻한다.
반증됨·철회됨	구체적 반례나 원인 확인 때문에 채택하지 않는 주장이다. 원기록은 남긴다.

과거 보고서의 `PASS` 는 항상 목적어를 동반해 읽는다. 예를 들어 `PASS_SCOPED_RAW_STATE_READBACK` 은 저장된 상태와 시간 집계의 읽기 검산이지, 연속 오차 정리나 생산 배포 승인 문구가 아니다. `roundoff_enclosed=false`, `HOST.wall_clock_admissible=false`, `FAIL_PRESERVED` 처럼 범위를 제한하는 상태도 삭제하지 않았다.

1.3 이 보고서의 공통 질문

모든 연구 단계를 관통하는 질문은 다음과 같다. **어떤 모델 구조와 출력 계약에서, 어떤 구현이, 어느 정확도까지, 얼마의 총비용으로 유효한 결과를 제공하는가?** 이 질문에는 방법 차수뿐 아니라 선형대수 backend, 비선형 target, 실제 출력 알고리즘, 실패 비용, 미분정보의 출처, rounding·모델 불확실성, 원자료의 보존 범위가 함께 들어간다.

2. 연구의 연속성과 단계별 산출물

단계	핵심 질문	실제 산출물	그 단계에서 확정하지 않은 것
I. 실행 환경·1 차 측정	첨부 Rust 로 현재 코드를 실제 실행할 수 있는가?	설치·서명·checksum 확인, 154 조건, matrix-free·A07 대조	142 개 구성요소의 공식 승격
II. 정밀 재측정	새 push 의 수치·시간 변화와 조밀 출력 정확도는?	old/new 교차 측정, CT 2,700 조건, allocation·Callgrind	모든 경로의 ±5% 비회귀
III. 외부 비교 정정, PR71	원본 DVODE 와 실제 오차를 맞추면 순위가 같은가?	native 공통 RHS, 756 조건 조사, 270 회 보정, 408 프로세스 정합 측정	보편적 솔버 순위
IV. Banded 개선, PR73	정확성을 바꾸지 않는 안전한 비용 절감은?	네 결함 수정, 고정폭 특화, exact LU cache	임의 stale Jacobian 재사용
V. 통합 이론	여덟 순차 stage 를 다른 실행 구조로 풀 수 있는가?	all-at-once homotopy, 공유 부분공간, causal majorant, Python 대조	범용 optimal router
VI. Native coupled, PR74	위 구조를 실제 JVP-only Rust step 으로 연결 가능한가?	companion crate, 19 시험, 800 프로세스	완성형 adaptive trajectory

단계	핵심 질문	실제 산출물	그 단계에서 확정하지 않은 것
VII. Residual-first, PR75	마지막에 쓰지 않는 미분과 실패한 후보 비용을 줄일 수 있는가?	opt-in residual-first, 제한된 JVP 예산, 33 시험	고정 cap 의 경제적 최적성
VIII. 중간점검, PR76	최신 후보를 외부 솔버와 다시 비교하면?	동일 호스트 39 설정·312 조건, 30 쌍 판정	미정합 few-mode 외부 순위
IX. 미분 없는 구조 탐색	물리식·대수기하·memory 로 JVP 자체를 우회할 수 있는가?	Volterra·반응 ideal·Schur·scalar 연속 인증·반례	일반 FVM 전체 인증
X. Hybrid, PR77	exact 축약과 국소 AD 를 같은 trajectory 승인에 연결 가능한가?	생성형 ODE, packet/structured/coupled 후보, 연속 반경, 외부 비교	범용 최적 선택·외부 BDF 우위
XI. 보존, PR78	코드·원자료·복구 증거를 재개 가능한 형태로 남겼는가?	네 ZIP·72 개 파일·Drive 이중 백업·검증 장부	생산 솔버 승급

이 순서는 단순한 개발 목록이 아니다. 초기에는 “정상 종료한 endpoint 를 얼마나 빨리 구하는가” 가 중심이었으나, 조밀 출력 반례와 외부 wrapper 결함을 거치며 “같은 실제 오차·같은 계산 의무의 비용” 으로 질문이 바뀌었다. 이후에는 여덟 순차 solve 의 실행 구조를 바꾸는 연구와, 모델의 구조로 solve 자체를 우회하는 연구가 분기했다. 하이브리드 단계는 이 두 가지를 같은 연속 출력 승인 아래 연결한 제한된 수렴점이다.

3. 측정·정확도·재현성을 비교하는 공통 언어

3.1 서로 다른 오차 지표를 하나로 합치지 않는다

초기 endpoint 비용 비교의 지표는

$$E_{\max} = \max_i \frac{|y_i(t_f) - y_{i,\mathrm{ref}}(t_f)|}{\max(|y_{i,\mathrm{ref}}(t_f)|, 10^{-10})}. \tag{3.1}$$

이에 경험적 reference spread 를 더한 값이 주로 10^{-5} 또는 10^{-7} 이하인 설정을 골랐다. 반면 이후 출력 격자의 요청 tolerance 기준은

$$R = \max_j \left[\frac{1}{N} \sum_i \left(\frac{y_i(t_j) - y_{i,\mathrm{ref}}(t_j)}{\mathrm{atol}_i + \mathrm{rtol}|y_{i,\mathrm{ref}}(t_j)|} \right)^2 \right]^{1/2} \tag{3.2}$$

와 같은 WRMS 였다. PR71 의 실제 정확도 정합에서는 고정된 문제별 scale s 를 사용해

$$E_{\mathrm{raw}} = \max_j \left[\frac{1}{N} \sum_i \left(\frac{y_i(t_j) - y_{i,\mathrm{ref}}(t_j)}{s + |y_{i,\mathrm{ref}}(t_j)|} \right)^2 \right]^{1/2}, \qquad E_{\mathrm{up}} = E_{\mathrm{raw}} + E_{\mathrm{reference}} \tag{3.3}$$

를 계산했다. 해당 캠페인의 $\mathrm{atol} = s\,\mathrm{rtol}$ 조건에서는 $R_{\mathrm{up}} = E_{\mathrm{up}}/\mathrm{rtol}$ 이다. 식 (3.1) 과 (3.3) 은 최대성분 norm 과 RMS 라는 차이까지 있으므로 수치 크기를 그대로 비교하면 안 된다. [A1, §§3–4; A2, §5; A3, §4]

`up` 라는 이름이 붙어 있어도 Radau tolerance ladder 나 Radau–LSODA 차이로 얻은 경험적 참조폭은 수학적으로 인증된 enclosure 가 아니다. 두 참조법의 공통 편향 가능성을 배제하지 않기 때문이다. 실제 참조 오차가 그 폭에 포함된다는 추가 전제가 있을 때만 삼각부등식으로 상계 해석을 할 수 있다.

3.2 다섯 가지 성공을 구분한다

실행 성공은 프로그램이 반환한 상태다. **Stage-root 성공**은 고정된 수치 stage 방정식을 충분히 풀었다는 판단이다. **Embedded 시간오차 통과**는 한 물리적 timestep 의 local estimator 기준이다. **출력 격자 정확도 통과**는 참조와 비교한 실제 누적·보간 오차에 대한 경험적 판단이다. **연속 오차 인증**은 명시한 모델·domain·연산 조건 아래 시간창 전체와 실제 반환값을 포함하는 증명 의무다.

이들은 서로 대체할 수 없다. `candidate.is_some()` 만으로 physical time 을 전진시키거나, `R_up>1` 만으로 local-tolerance 솔버의 구현 오류라고 판정하면 안 된다. Netlib DVODE 도 실제 global error 가 local tolerance 를 초과할 수 있음을 명시한다. [A3, §§4, 6; R74; E2]

3.3 네 가지 비용도 구분한다

비용 종류	예시	직접 비교가 가능한 조건
고정 h stage-root latency	PR74·PR75	같은 target 또는 정당화된 target 변환, root·embedded 정확도 조건
구간 전체 적분 시간	초기 캠페인·PR71·PR76	동일 IVP·출력 schedule·달성 오차·호스트
연속 인증 trajectory 비용	PR77	endpoint·curve·certificate·반환 반경이라는 계산 의무까지 일치
subprocess 또는 계측 비용	초기 MF 초 단위, Callgrind, allocation	포함되는 setup·직렬화·프로세스 시작·계측 범위를 일치

단위가 모두 ms 여도 비용의 목적어가 다르면 나눌 수 없다. 특히 PR77 의 타이머에는 native `run()` 내부의 `serde_json::to_value` 를 통한 proof·통계 객체 구성이 포함된다. 외부 문자열 출력과 result destruction 만 제외됐다는 후속 확인을 성능 해석에 반영한다. [A5, §9]

3.4 통계 단위와 동일성

내부 적분 수천 회를 독립 표본 수천 개로 세지 않는다. 각 캠페인이 선택한 fresh-process median 또는 paired block 이 재표집 단위다. 일부 단계는 pointwise 구간만, 다른 단계는 Bonferroni-family 구간을 제공했다. 공유 VM 의 CPU frequency·가상화·자기상관과 적은 블록 수에 관한 불확실성은 좁은 bootstrap 구간만으로 제거되지 않는다.

소스 바이트 동일성, canonical 내용 동일성, 관측된 binary64 결과 동일성, 수학적 target 동일성, 과학적 동등성도 분리한다. `8a8bd40` 과 후속 `a21d573` 의 Rust 입력 소스가 같더라도 build-time revision 을 내장한 바이너리 바이트까지 같다는 뜻은 아니다. Raw-U 와 native-K 는 이론적 관계가 있어도 coefficient 변환의 rounding 때문에 비트 동일성을 가정하지 않는다. [A2, §1; S05, §1]

4. 초기 실행 환경 복구와 첫 벤치마크

4.1 실제 설치와 빌드

첨부 Rust archive 의 SHA-256 과 `.asc` 서명을 검사하고 Linux 실행환경에 Rust/Cargo 1.94.1, LLVM 21.1.8 을 설치했다. `RUSTCORE(9).zip` 은 VigilODE 자체가 아닌 별도 프로젝트였으며, 그 vendor 에서 lock 과 일치하는 155 개 패키지를 재사용하고 누락된 36 개를 공식 crate archive 에서 받아 checksum 을 검사했다. 실제 프로그램 compile/run 도 확인했다. [A1, §§1–2]

측정 대상은 `claude/nice-dijkstra-208xq0` 의 `ccaeaa50beb4580505a36d25ccb919ec3ca38c7f` 였다. 해당 최신 commit 은 142 개 구성요소 분류를 위한 manifest·사전등록 추가였고 적분 엔진 변경은 아니었다. 따라서 이 측정으로 142 개 전체의 공식 triage 를 완료했다고 볼 수 없다.

원래 CGU=16 measurement 설정의 CLI link 에서 undefined hidden symbol 오류가 발생했다. CLI 에 `-C codegen-units=1` 을 적용하자 빌드됐으며, dependency profile 은 CGU=16 이었다. 이것은 실행에

필요한 빌드 조건의 변경이지 특정 rustc/LLVM 결함의 원인을 확정한 결과가 아니다. Host 는 AMD EPYC 9V74 가 노출된 공유 KVM 이며 CPU 2 affinity, Rayon·BLAS·OMP·MKL 각 1 스레드를 사용했다.

4.2 실행 규모와 모델

다섯 문제는 Robertson 3 변수 $[0, 40]$, HIRES 8 변수 $[0, 321.8122]$, van der Pol $\mu = 1000$ 의 2 변수 $[0, 2000]$, Brusselator 50·200 개 공간점의 100·400 변수 $[0, 10]$ 였다. Robertson·HIRES 의 atol $= 10^{-4}$rtol, 나머지는 atol $=$ rtol 이었다. rtol $= 10^{-3}, \dots, 10^{-9}$, $h_0 = 10^{-6}$, 시작점·끝점 출력을 사용했다.

Direct/fast/small/banded/router 의 154 조건을 조건당 warmup 1 회·측정 7 회로 실행하여 1,078 회 timed 적분을 얻었다. Matrix-free 는 24 조건, 조건당 적분 3 회였다. 표적 Rust 시험은 40 passed·0 failed·2 ignored 였고, Diffsol A07 은 별도의 182 행을 생성했다. 전체 workspace 통과라는 주장은 하지 않았다. [A1, §2]

4.3 같은 끝점 목표 아래의 내부 경로 차이

아래는 $E_{\max} + \rho \le 10^{-5}$ 를 만족한 정상·반복일치 설정 가운데 1 차 분석이 선택한 값이다. 이 단계는 허용된 행 중 시간 median 이 가장 짧은 행을 선택했다. 뒤의 V2 는 정확도를 만족하는 가장 느슨한 측정 tolerance 를 먼저 선택하는 규칙을 사용하므로 선택 정책도 같지 않다. 단위는 ms/구간 적분이다. [A1, §3; A2, §4]

문제	Generic direct	Fast direct	특화 경로	특화 시간	Generic/특화
Robertson	0.228635	0.023524	fast-small	0.018197	12.56
HIRES	1.199540	0.189898	fast-small-static	0.140957	8.51
van der Pol	3.228738	0.313410	fast-small	0.208466	15.49
Brusselator $N = 100$	18.581235	2.744597	fast-banded	1.730129	10.74
Brusselator $N = 400$	409.833040	25.659520	banded-slices	7.164704	57.20

Generic/fast 의 35 비교와 generic/특화의 119 비교에서 accepted/rejected 수가 같았다. 예를 들어 $N = 400$, rtol 10^{-6} 의 generic·fast 는 82 accepted, 10 rejected, 92 LU, 736 direct solve 였다. 따라서 큰 시간 차이는 timestep 을 적게 수행한 결과가 아니다. Workspace 재사용, transformed stage 계산으로 일부 JVP 제거, 진단 객체·residual 작업 축소, 다른 LU 구현, retry 시 base RHS/Jacobian 재사용이 원인 후보였으며, Jacobian build 는 92 회에서 82 회로 줄었다. 원인별 wall-time 기여율을 분리한 profiling 은 하지 않았다.

$N = 100 \to 400$ 에서 generic direct 시간은 22.06 배, dense-storage fast 는 9.35 배, banded-slices 는 4.02 배 증가했다. 이는 고정 bandwidth 의 구조 활용을 지지하지만 모든 2 차원 격자의 $O(N)$ 성능을 뜻하지 않는다. 조건별 IQR/median 의 중앙값은 6.87%, 최대 56.68% 여서 작은 특화 구현 사이의 수% 차이로 안정적인 순위를 정하지 않았다.

4.4 조밀 출력과 matrix-free 에서 드러난 문제

정확해가 $y(t) = \sin t$ 인

$$y' = -10^6(y - \sin t) + \cos t + 10(y - \sin t)^2, \quad y(0) = 0$$

를 솔버에 정확해 callback 없이 제공했다. $[0, 1]$, uniform101, rtol 10^{-8}, atol 10^{-11}, 순차 GMRES 조건에서 dense Off 는 6/0 accepted/rejected 와 최대 정규화 오차 1801.6160, Enforce 는 17/24 와 1.4986 이었다. Enforce 가 큰 개선을 만들었지만 오차 1 이내에는 들지 못했다. 채택된 dense estimate

가 최대 0.9217 이었다는 사실과 실제 출력오차 1.4986 의 차이가 후속 continuous-defect 연구의 직접적인 동기였다. [A1, §4]

Matrix-free 의 rtol 10^{-6} 대조는 다음과 같다. 이 값은 **초/subprocess** 이며 적분 3 회·초기화·직렬화가 포함된다. 위 표의 ms/적분과 나누면 안 된다.

차원	mf-legacy-v2	mf	mf-into	mf-ls-ws
100	0.565958	1.870349	0.566407	0.566025
400	4.229644	7.550354	4.077425	4.180307

일반 MF 도 persistent workspace 를 사용하고 있었다. 중요한 차이는 매 Arnoldi column 에서 사용하지 않을 Hessenberg-prefix QR solve 를 반복하는가, cycle 끝에서 한 번만 수행하는가였다. JVP·orthogonalization counter 가 같아도 이 비용은 달랐다. $N = 400$, rtol 10^{-4} 에서는 모든 경로가 최종 완료했지만 linear failure 29 회, local-error failure 4 회, 64/33 accepted/rejected, linear iteration 75,400 회였다. 더 엄격한 rtol 10^{-6} 은 64,120 회여서 outer tolerance 를 느슨하게 하는 것이 내부 일을 줄이지도 않았다. [A1, §5]

4.5 최초 Diffsol A07 비교의 정확한 위치

Robertson, uniform101, rtol 10^{-8}, atol 10^{-11} 에서 A07 의 182 행은 실행·식별 검증을 통과했다. 그러나 최대 출력-grid WRMS 는 native fast 0.337528, Diffsol BDF 3.302700 으로 정확도 gate 가 달랐다. Fresh-process median 은 0.286035/0.263787 ms, 반복 ABBA/BAAB median 은 0.110152/0.132139 ms 였지만, 동일 달성 오차에서의 순위로 채택하지 않았다. [A1, §6]

할당 요청은 439/1,039 회, 누적 요청 bytes 는 42,675/116,634 였다. 이는 peak RSS 나 step hot-loop zero-allocation 의 증거가 아니다. Validator exit 4 는 이 캠페인의 정확도 정합 실패였으며 compile 실패나 적분 중단이 아니었다. 기록형식·성공 gate·다른 tolerance 의 counter 혼합에 관한 분석기 결함도 분리해서 수정·회피했다.

5. 정밀 재벤치마크: 수치 동일성, CT 2,700 조건, 계측 비용

5.1 실측 커밋과 최신 HEAD 의 구분

두 번째 측정은 최초 `ccaeaa50...` 와 새로 빌드한 `8a8bd409b1d96c7b7421f496d229e119fd3d284e` 를 교차 실행했다. 작업 중 브랜치 HEAD 는 `a21d573980f66fe02891bd216153443b5352f921` 로 이동했지만 후속 세 커밋은 Python CT 도구·정책·자료·ignore 규칙의 변경이었다. Rust 소스·Cargo 설정·lock·생산 코드 입력자료의 동등성은 확인했고, 최신 Python 도구는 별도로 실행했다. 시간·할당·명령어 수를 최신 HEAD 에서 새로 빌드한 수치로 재명명하지 않았다. [A2, §1]

시간 측정은 154 조건, 120 fresh processes, timed 18,480 회와 warmup 6,160 회였다. ABBA·BAAB 각 5 개씩인 10 개 교차 블록을 재표집했다. Wired 경로 73 개, 적용 가능한 242 조합을 확인했으며, CT 는 12 문제·20 경로의 적용 가능한 150 조합에 tolerance 3 개·출력 격자 2 개·초기 step 배율 3 개를 적용한 2,700 조건이었다. 24 개 독립 참조해, 150 조건 재현성 대조, 84 allocation 조건, 14 Callgrind 대표 조건도 별도로 수행했다.

시간·정확도용 measurement binary 와 fat-LTO bench 계측 앱은 다른 빌드였다. 공유 VM 의 메모리 한도는 8 GiB 였고 `HOST.wall_clock_admissible=false` 였다. 따라서 여기의 시간구간은 저장소가 요구하는 전용 호스트 성능 승격 조건을 충족한 것이 아니다. [A2, §2]

5.2 버전 간 변화와 버전 내부 경로 차이는 별개다

154 조건 모두에서 JSON 으로 복원한 끝점 binary64 bits, 여섯 counter, accepted/rejected 수가 일치했다. 각 조건의 old/new 를 합친 40 프로세스 관측에도 동일했다. 그러나 22 개 문제·경로 그룹의 시간구간 중 ±5% 안에 완전히 들어간 것은 7 개뿐이고, 나머지 15 개는 미결정이었다. 5% 를 넘는 증가나 감소를 구

간 전체가 지지하는 그룹은 없었다. **확인된 큰 회귀가 없다는 것과 모든 경로의 ±5% 비회귀를 보장한다는 것은 다르다.** [A2, §3]

새 버전 내부에서 같은 끝점 목표를 만족하는 가장 느슨한 rtol 을 먼저 고른 결과는 다음과 같다. 단위는 ms/구간 적분이며, 시간 보고의 원전은 새 버전 20 개 프로세스의 중앙값 요약이다.

문제	rtol	Base	Fast	특화 시간	Base/특화
Robertson	10^{-5}	0.233718	0.024020	0.018763	12.46
HIRES	10^{-5}	1.199571	0.166224	0.143731	8.35
van der Pol	10^{-6}	3.236180	0.275580	0.216012	14.98
Brusselator 100	10^{-6}	18.872362	2.637861	1.826857	10.33
Brusselator 400	10^{-6}	387.745482	26.418277	7.335145	52.86

약 52.86 배는 이 버전의 base 와 banded-slices 사이의 차이다. 첫 캠페인의 409.833 ms 를 다른 실행의 7.335 ms 와 나누어 버전 개선률을 만드는 것은 허용되지 않는다. [A2, §4]

5.3 정확도 분포와 조밀 출력 반례

CT 2,700 조건은 2,639 정상 종료, 54 지원 범위에 따른 거부, 7 solver 실패로 분류됐다. 정상 종료를 분모로 하면 $R_{\mathrm{up}} \le 1$ 은 2,329 개 (88.25%), 10 이하는 2,580 개 (97.76%), 100 이하는 2,626 개 (99.51%) 였다. 전체 계획을 분모로 한 정상·정확도 동시 충족은 2,329/2,700=86.26% 였다. 초기 step 배율 세 개가 모두 성공한 875 그룹 중, 세 배율의 최악값도 1 이내인 것은 760 그룹이었다. [A2, §5]

비선형 Prothero–Robinson, uniform101, rtol 10^{-8}, atol 10^{-11}, $h_0 = 10^{-6}$ 에서는 다음 차이가 있었다.

경로	실제 R_{up}	Accepted / Rejected
관측 시각에 step 을 맞춘 seq-direct	6.63×10^{-7}	106 / 0
seq-direct-dense-off	1751.99	11 / 0
seq-direct-dense-enforce	1.0933	21 / 23

이 표의 h_0 는 첫 별도 probe 의 10^{-3} 과 다르다. 첫 probe 를 같은 조건으로 다시 실행했을 때 Off 1801.6160, Enforce 1.49857 은 이전 raw 결과와 바이트 단위로 같았다. 두 결과를 불일치한 반복측정처럼 취급하지 않는다.

`fast-ctl-pi` 는 적용된 144 조건 모두 $R_{\mathrm{up}} \le 1$, 최대 약 0.8860 을 보였다. 이는 적용 범위 내 정확도 근거이지 전체 문제군에서의 승격이나 속도 우위가 아니다. 일곱 solver 실패 중 여섯은 manufactured-vector-32 의 초기 step 2 배 조건에서 `seq-mf-gmres` 의 inner-forcing norm 오류, 하나는 complex-Dahlquist-32 의 엄격한 uniform101 조건에서 fused exponential 경로의 적분 한계였다. 지원 범위 거부 54 개와 수치 실패 7 개를 합쳐 같은 실패율로 해석하지 않는다.

5.4 메모리와 명령어 수의 서로 다른 효과

Advection–diffusion $N = 96$, rtol 10^{-6}, uniform101 의 대표 대조에서 다음을 관측했다. 두 bench 경로의 출력은 같고 $R_{\mathrm{up}} \simeq 0.2013235$ 였다. [A2, §6]

지표	mf	mf-into	해석
할당 요청 횟수	373,558	11,922	약 31.33 배 감소
누적 요청 bytes	610,386,479	42,976,927	약 14.20 배 감소
Window peak bytes	246,576	243,000	약 1.45% 감소

지표	mf	mf-into	해석
Warm Callgrind 명령어 수	5,899,710,427	1,769,147,834	약 3.33 배 감소

할당 횟수 31 배 감소를 최대 메모리 31 배 감소 또는 시간 31 배 가속으로 바꾸면 안 된다. Window peak 는 계수 구간 시작점에 대한 요청량 증분이고 전체 프로세스 RSS 가 아니다. Callgrind 는 적분뿐 아니라 CT 실행기의 결과 병합·수명관리·반복 비교도 포함한 차분이다. 14 대표 조건의 계측·빌드 연결은 통과했지만, 실제 bench $R_{\text{up}} \le 1$ 까지 만족한 것은 12 개였으며 PR dense Off/Enforce 는 제외됐다.

5.5 분석 도구의 결함과 수정 상태

최신 Python 도구에서는 Rust parity 형식 연결이 모델이 있는 7 문제에서 통과했고, 5 문제는 NumPy 모델 부재로 미검사였다. Radau/LSODA 및 exact 출력 연결은 24 참조 설정 모두 생성에 성공했다. Mass-affine 두 격자를 제외한 22 개는 독립 참조와 상태·폭 bits 가 같았다. [A2, §7]

동시에 세 가지 결함은 재현됐다. Mass 를 지원하는 경로가 반드시 mass 문제만 받아야 한다고 해석하여 정적 적용성 조합 1,191 개 중 496 개를 제외했다. 출력시각 101 개에 상태 하나만 남긴 합성 사본을 허용하여 원래 $R_{\text{up}} = 1751.99$ 인 행을 0 으로 계산했다. 실제 Rust 의 `error_kind`, `segments`, `segments_completed` 를 예상하지 않은 schema key 로 거절했다. 부분 상태 사례는 분석기에 대한 합성 반례이며 실제 solver 가 그런 성공 행을 출력했다는 관측은 아니다. 실제 성공 2,639 행은 별도 길이·차원·시간 검사에 통과했다.

표적 Python 90 개 메서드 중 역사 재현 2 메서드의 6 개 하위 실패가 남았고, 나머지 88 메서드에는 실패가 없었다. 새 triage 시험 7 개에도 실패가 없었다. Rust DP-02 4 개, triage CLI 21 개 통과·4 ignored 는 표적 시험이지 전체 workspace 완료가 아니다. 이로써 **수치 솔버와 그 결과를 승인하는 분석기 역시 별개의 검증 대상**이라는 점이 명확해졌다.

6. 외부 비교의 정정과 엄격한 native 3 자 비교: PR71

6.1 초기 SciPy VODE 결과는 왜 재해석됐는가

초기 별도 비교는 SciPy 1.17.0 VODE wrapper 를 사용했다. Robertson uniform101 에서 요청 tolerance 단위 오차가 약 128.502, 매우 큰 step·callback 수가 기록됐고, van der Pol 은 step ceiling 에 이르렀다. 그 원보고서도 호스트 차이와 실제 오차 불일치 때문에 3 자 속도 순위를 미결정으로 두었다. 이후 SciPy 1.17 의 C 이식 wrapper 가 full analytical Jacobian 을 전치해 소비하는 문제가 upstream PR #24934 에서 확인되면서, 이 관측을 **원본 Fortran VODE 알고리즘의 열세로 귀속하는 해석은 철회**됐다. [S12; A3, §1; E1]

정정은 원자료를 지우는 방식이 아니었다. C/F 배열, 전치 대조군, 내부 FD, banded 대조를 별도로 실행했다. Robertson 의 전치 대조군에서는 step 증가가 크게 줄었지만 HIRES·van der Pol 의 추가 dense-path 이상은 남았다. 알려진 전치 버그와 남은 이상의 미확정 원인을 같은 결함으로 단정하지 않았다.

6.2 공정성 계약을 강화한 비교

주 비교는 Netlib 원본 Fortran DVODE 와 Diffsol 0.16.1 BDF/FaerLU dense, VigilODE fast 및 banded-slices 를 공통 native 실행기에 연결했다. 세 솔버는 같은 Rust RHS·해석 Jacobian 함수를 사용하고 DVODE 는 ISO_C_BINDING bridge 를 거쳤다. Python callback 비용을 VODE 에만 부과하지 않았다. 단, 실제 API dispatch·FFI·초기화·출력보존 비용은 각 구현의 사용 비용으로 남았다. DVODE 는 작은 문제 MF21 dense, 공간 문제 MF24 banded 였다. [A3, §§1–2; E2]

주 조사는 756 accuracy cells, 1,152 timing processes 였다. 후속 정합 캠페인은 270 calibration solves, 408 timing processes, 4,896 timed integrations 를 수행했다. Nominal h_0 에서

$$\tfrac{1}{2}\tau_* \le E_{\text{up}} \le \tau_*, \qquad \max_{h_0 \in \{0.5,1,2\}\, h_{0,\text{nominal}}} E_{\text{up}} \le \tau_*, \qquad E_{\text{reference}} \le 0.1\tau_* \tag{6.1}$$

를 만족하도록 solver 별 rtol 을 시간과 독립적으로 조정했다. 열 개 문제/격자 그룹 모두 정합에 성공했다. 주 조사와 정합 실행의 host 가 달라 시간 표본을 합치지 않았다.

6.3 실제 달성 오차를 맞춘 결과

단위는 ms/적분이다. 한 행의 nominal 오차들은 최대 2 배 범위지만 다른 행의 τ_* 까지 같은 것은 아니다. [A3, §3]

문제	출력	Fast	Banded	Diffsol dense	DVODE
Robertson	끝점	0.037920	해당 없음	0.108376	0.100469
Robertson	101 점	0.089863	해당 없음	0.116443	0.117109
HIRES	끝점	0.281925	해당 없음	0.536127	0.479670
HIRES	101 점	0.373237	해당 없음	0.590071	0.533075
van der Pol	끝점	0.265004	해당 없음	0.983506	0.753888
van der Pol	101 점	0.331527	해당 없음	1.152068	0.933415
Brusselator 100	끝점	2.819666	1.999169	5.156916	0.943856
Brusselator 100	101 점	3.883919	2.759208	5.424570	1.562516
Brusselator 400	끝점	27.411388	7.900871	141.289589	3.610742
Brusselator 400	101 점	44.820667	12.830880	131.609302	6.154194

작은 세 stiff 계에서는 fast 가 유리했다. 반면 banded 를 양쪽에 허용해도 DVODE 는 네 공간 문제/출력에서 VigilODE banded 보다 약 1.77–2.19 배 짧았다. Diffsol 의 큰 공간 비용은 dense backend 의 결과이지 Diffsol 전체나 Rust 언어의 한계가 아니다.

42 개 사전 지정 pair, 12 개 paired blocks, 100,000 bootstrap 재표집과 family 보정, exact sign test 가 저장됐다. 예를 들어 Robertson endpoint 의 Diffsol/fast 는 2.8575, 보정 구간 [2.8460, 2.8740], DVODE/fast 는 2.6487, [2.6327, 2.6571] 이었다. Brusselator 400 endpoint 의 banded/DVODE 는 2.1886, [2.1821, 2.1956] 이었다. 구간은 해당 공유 host·선택 설정에 조건부인 진단이며 보편적 생산 성능 보증이 아니다.

6.4 Step 수만으로 비용을 설명할 수 없는 이유

Brusselator 400 endpoint 에서 VigilODE banded 는 100 accepted, Jacobian 100, LU 108, RHS 856 회였고 DVODE 는 272 step, Jacobian 5, LU 31, RHS 313 회였다. 더 적은 macro-step 을 쓰는 전자가 더 짧지 않았다. BDF 의 재사용과 Rosenbrock stage 당 비용의 차이가 관측과 일관되지만 원인별 시간 비중까지 측정한 것은 아니다. [A3, §5]

이 결과는 “BDF 의 Jacobian 재사용을 RODAS5P 에 그대로 이식하자” 는 결론을 정당화하지 않는다. 근사·지연 Jacobian 을 허용하는 방법은 별도의 차수·안정성 조건이 필요하다. 이후 PR73 이 exact shifted-matrix identity 라는 보수적 조건부터 연구한 이유다. [R73; B2]

6.5 실패를 보존한 독립 읽기 검산

Fortran callback 의 band 경계, NumPy int32 직렬화, adapter 문법 오류를 각각 수정했다. 최종 적분은 끝났으나 최초 auditor 는 Diffsol 18 행의 초기 출력과 y_0 의 비트 불일치로 멈췄다. 최대 절대차는 2.22×10^{-16}, 요청오차 단위 기여는 1.17×10^{-10} 이었다. 원래 값을 덮어쓰지 않고 오차에 남긴 채 별도의 floating-consistency 검사를 수행했다. 원래 exact-bit FAIL 도 유지했다. 최종 상태는 `PASS_SCOPED_READBACK_WITH_INITIAL_ROUNDING` 이다. [A3, §6]

678 행, 189 개 상태 객체, 42 paired median, 225 개 보존 파일의 hash 를 확인했고 Robertson·HIRES 보존량의 최대 잔차는 4.22×10^{-15} 였다. 같은 작성자의 다른 계산 경로이므로 독립 연구자 심사와 동일하

지 않다. 주 screening 의 raw gzip 두 개가 용량 필터로 빠졌다는 한계와, 후속 정합 실행에서는 전체 상태·개별 시간 표본을 content-addressed 로 보존했다는 개선도 함께 남겼다.

7. Banded 정확성 수정과 exact LU 재사용: PR73

7.1 실제 결함과 반증된 의심

원본 standalone band solve 에서 네 문제가 재현됐다. 비유한 RHS 에 `Ok(NaN)`, 비유한 상삼각 원소에도 `Ok(NaN)`, 유한 입력이지만 표현 불가능한 해에 `Ok(Inf)`, 극단적인 차원·bandwidth 계산에 overflow panic 이 발생했다. 입력·출력 finite 검사와 checked size arithmetic 으로 수정했고, 작은 pivot 이어도 해가 표현 가능한 경우는 계속 허용했다. [R73]

처음 의심했던 callback 실패 후 partial snapshot 재사용은 원본의 네 retry 시험이 통과했고 완전한 Err 분기에서 유효성을 지우고 있어 반증됐다. 그 코드는 수정하지 않았다. 또한 첫 benchmark adapter 의 JVP 미등록은 수치 불일치가 아니라 적분 전 `ADAPTER_CONSTRUCTION_FAILURE` 로 재분류했다.

7.2 Exact reuse 의 충분조건

Fast 구현이 사용하는 시간 역수 단위의 행렬을

$$\mathcal{W}(h, J) = \frac{1}{\gamma h} I - J$$

라고 쓰면

$$\mathcal{W}_1 - \mathcal{W}_0 = \left(\frac{1}{\gamma h_1} - \frac{1}{\gamma h_0} \right) I - (J_1 - J_0). \tag{7.1}$$

$J_1 = J_0$ 이어도 shift 가 바뀌면 같은 LU 를 정확해로 재사용할 수 없다. 새 cache 는 계산된 유한 shift 와 저장한 Jacobian 전항목의 binary64 bits 가 같고, 이전 factorization 이 성공하여 factor 도 유한할 때만 hit 한다. Failed factorization 은 key 를 먼저 무효화한다. 새로운 accepted state 의 RHS·Jacobian callback 자체는 생략하지 않는다.

같은 finite matrix 에서 같은 pivot tie·산술 순서를 사용하고 solve 가 factor 를 변경하지 않는다는 가정 아래, factor 재계산 생략은 stage·embedded error·step 선택을 바꾸지 않는다. 이는 현재 target 을 푸는 exact 재사용의 충분조건이지, stale Jacobian 을 허용하는 새 Rosenbrock/W 방법이 아니다. [R73; B2]

7.3 고정폭 특화와 실측

`FixedWidth2` 는 lower=upper=2 의 factor/solve 를 같은 산술·pivot 순서로 특화하고 다른 폭에서는 Slices 로 돌아간다. `FixedWidth2ExactCache` 는 그 위에 exact cache 를 더한 opt-in 이다. 432 행에서 324 개 parent/new 의 상태·출력시각 bits 가 같았고, cache 관련 counter 외 작업량도 일치했다. rtol 세 개, h_0 배율 세 개, 50·127·200 cell 크기를 포함했다. 시간은 별도의 rtol 10^{-6}, 12 블록, 576 프로세스·3,744 timed 적분에서 측정했다.

다음은 $N = 400$ 의 ms/적분이다. [R73]

문제·출력	이전 Slices	새 Slices	FixedWidth2	Exact cache
Brusselator 끝점	7.851086	7.718258	7.485703	7.651167
Brusselator 101 점	11.861624	11.703437	11.302102	11.567599
선형 diffusion 끝점	0.843415	0.828938	0.804731	0.825487

문제·출력	이전 Slices	새 Slices	FixedWidth2	Exact cache
선형 diffusion dyadic129	11.347407	11.213596	10.846163	10.167443

비선형 문제의 FixedWidth2 감소는 parent 대비 약 4.7–5.3% 였지만 같은 새 binary 의 Slices 도 소폭 짧았다. 전체 차이를 고정폭 특화 하나에 귀속할 수 없고, 같은 binary 의 추가 감소는 대략 3% 였다.

선형 dyadic129 에서는 LU 가 134 회에서 8 회로 줄고 126 회 재사용됐으며 Jacobian 134 회는 유지됐다. 전체 시간 감소는 10–11% 였다. 비선형 Brusselator 와 선형 endpoint 에서는 cache hit 가 0 회였고, 비선형에서는 cache 가 FixedWidth2 만 쓴 경로보다 2–3% 느렸다. 상수 J 도 변하는 h 때문에 재사용을 보장하지 않는다는 부정 결과다.

양성 대조인 dyadic129 는 출력 시각에 step 을 맞춰 반복 행렬을 많이 만들었다. 경험적 오차는 약 1.6×10^{-13} 으로 입력 rtol 10^{-6} 보다 훨씬 작았고 endpoint 10 attempts 에 비해 134 attempts 였다. Cache 의 이득은 출력정책으로 생긴 과잉 적분 비용을 줄인 측면을 포함한다. Key 는 추가 $N(l+u+1)$ 개의 u64 를 사용하므로 폭 2, $N = 400$ 에서 16,000 bytes 이며 이는 전체 peak memory 가 아니다.

7.4 검증 범위

회귀 9 개와 관련 내부 17 개가 통과했다. 전체 workspace 는 660 초 timeout, exit 124 였고 PASS 로 바꾸지 않았다. 별도의 역사 provenance 장부 `L-0124` 의 두 파일 hash 불일치도 해당 패치가 고쳤다고 주장하지 않았다. 후속 복구에서는 기존 상태·오차·36 개 paired 비교의 원자료 연결을 다시 확인했지만 새 적분·새 시간 측정으로 세지 않았다. 이 노드의 성과는 검증된 결함 수정과 선택형 비용 개선이며, DVODE 의 공간 문제 우위 해소나 기본 커널 승격은 아니다. [R73; R73R]

8. 통합 이론: matrix-free·동시 stage·homotopy

8.1 실행 구조가 분리돼 있던 상태

기준 `c03ca90f...` 에서 router 는 declared band/small dense/explicit Jacobian/JVP 에 따라 경로를 골랐지만 homotopy·공유 Krylov·stage 병렬의 통합 dispatcher 는 아니었다. `ParallelExecution` 은 실제 local Rayon 병렬 작업을 제공했지만 common-W `solve_rows` 는 한 mutable workspace 로 순서대로 풀었다. 기존 homotopy path 는 explicit common-W 와 LU 를 요구했다. “batch”, “matrix-free”, “parallel” 기능이 각각 있다는 것과 동일한 적분 경로에 결합됐다는 것은 달랐다. [S05, §1]

8.2 같은 raw-U target 의 전체 stage 방정식

자율·항등질량 ODE $\dot{y} = f(y)$, 필요한 domain 에서 $f \in C^1$, $h > 0$ 를 가정한다. $J_0 = f'(y_n)$ 은 macro-attempt 안에서 고정한다. 이 절의 $W = I - h\gamma J_0$ 는 무차원이며 앞 절의 $\mathcal{W}$ 와 $W = \gamma h \mathcal{W}$ 관계다. Raw stage U_i 는 상태와 같은 단위다. 엄격한 하삼각 coefficient A, C 와 $s = 8$ 에 대해

$$F_i(U) = WU_i - \gamma \sum_{j<i} C_{ij} U_j - h\gamma f\left(y_n + \sum_{j<i} A_{ij} U_j\right) = 0. \tag{8.1}$$

$f(y_n + x) = f_0 + J_0 x + N(x)$, $D = I_s \otimes W$, $B = \gamma C \otimes I + h\gamma A \otimes J_0$, $b = \mathbf{1}_s \otimes (h\gamma f_0)$, $R_i(U) = N(\sum_{j<i} A_{ij} U_j)$ 로 두면

$$F(U) = (D - B)U - b - h\gamma R(U).$$

통합 homotopy 는

$$H(U, \lambda; \theta) = [D - \eta(\lambda) B]U - b - \lambda h\gamma R(U), \quad \eta(\lambda) = \theta + \lambda(1 - \theta). \tag{8.2}$$

$\lambda = 1$ 에서는 모든 θ 가 같은 raw target 으로 돌아온다. $\theta = 0$ 은 decoupled seed, $\theta = 1$ 은 affine stage coupling 을 먼저 처리한 seed 다. 후자의 생성 비용은 공짜가 아니다.

8.3 고정 W 에서의 stage-root 구조

R_i 는 이전 stage 에만 의존하므로 H_U 는 대각 block 이 모두 W 인 하삼각 block matrix 다. 따라서

$$\det H_U = \det(W)^8. \tag{8.3}$$

고정된 가역 W 와 유효 domain 에서 λ, U 변화만으로 stage-root fold 가 생기지 않는다. 이는 condition number·path curvature 가 작다는 뜻이 아니며, domain 이탈·overflow·variable mass 나 별도의 물리적 equilibrium root 에는 적용하지 않는다. 각 stage 가 순서대로 유일하게 결정된다는 구조에서 나온 제한된 명제이지 일반 homotopy 의 무분기 정리가 아니다. 정확한 tangent 는

$$H_U \frac{dU}{d\lambda} = (1-\theta)BU + h\gamma R(U) \tag{8.4}$$

를 만족한다. Truncated partial-linear predictor 는 이를 근사할 수 있지만 nonlinear derivative 까지 포함한 exact tangent 로 재명명하지 않는다. [S05, §2]

Affine 경우 $T = D^{-1}B$ 는 $T^8 = 0$ 이지만 $\rho(T) = 0$ 이 1–2 회 반복의 실용 수렴을 보장하지 않는다. Raw coefficient 의 scalar $hJ = -100$ 대조에서는 $\|T^3\|_\infty \simeq 81.89$ 의 transient 가 있었다. 또 $hJ = -0.01$ 에서 raw-U splitting norm 은 약 68.676, 변환한 K 표현은 약 0.07838 이었다. 좌표의 선택은 중요하지만 좌표별 norm 을 직접 비교해 물리 출력오차를 단정하지 않는다.

$(I+T)(I+T^2)(I+T^4) = \sum_{k=0}^{7} T^k$ 는 작은 명시적 stage matrix 의 doubling 을 허용한다. 그러나 black-box matrix-free $T^2 v$ 는 여전히 T 두 번의 비용을 지불한다. 작은 certificate 의 logarithmic algebraic depth 를 큰 연산자의 무료 병렬 깊이로 옮길 수 없다.

8.4 공유 부분공간과 원래 차원의 승인

JVP 로 V_m 과 $J_0 V_m$ 을 만들고 $U \approx (I_s \otimes V_m)z$ 로 표현하면 $8m$ 차원의 작은 전체 stage 문제를 풀 수 있다. 마지막에는 원래 N 차원의 residual 을 검사하고, nonlinear remainder 가 basis 밖으로 빠지면 그 방향을 추가한다. RHS rank 나 projected residual 만으로 승인하지 않는다.

원래 W 가 가역이어도 $V^T W V$ 는 특이할 수 있다. 예를 들어 $W = \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right)$, $V = e_1$ 이면 $\det W = 1$ 이나 $V^T W V = 0$ 이다. 작은 선형계의 가역성·조건도 별도로 검사해야 한다. [S05, §3]

범용 full-space stage Newton–Krylov, parallel exponential Rosenbrock·peer/W family, 기존 sequential 보호 경로는 함께 남긴 후보였다. 후자의 exponential/peer 는 기존 RODAS5P 순서 변경과 다른 방법이며 별도의 embedded·dense·switch 계약이 필요하다. 이 이론 탐색에서 실제 method switching 을 실행했다고 주장하지 않았다.

8.5 경량 실험의 긍정 결과와 반례

총 30 개 homotopy 대조에서는 $f_i = a_i y_i + \epsilon y_i^2$, $a = (-1, -10, -100, -1000)$ 를 사용했다. 모든 설정이 종료했고 raw target 출력 차이는 WRMS 0.0001594 미만이었다. 일부 affine-seed 대조는 seed 를 oracle 로 제공해 그 비용을 제외했으므로 전체 속도비의 근거가 아니다. [S05, §6]

12,288 변수 세 spectral-group 예제에서는 oracle 없이 JVP 로 $m = 3$ 을 만들고 24 차원 stage-root 를 풀었다. 실제 동시 stage task 는 4 개, $\lambda = 1$ 직접 보정은 작은 Newton solve 2 회·stage batch 3 회, full relative residual 6.64×10^{-15}, 참조 stage-root 출력 WRMS 1.28×10^{-8} 이었다. Basis JVP 3 회 외에 stage derivative 72 vector-equivalents, RHS 24 stage-vectors, full residual 검사가 있었다. “전체 JVP 3 회” 라는 축약은 잘못이다.

같은 문제에 네 λ 구간을 강제하면 small solve 6 회·stage batch 10 회·derivative 240 vector-equivalents 로 늘었다. 작은 다른 대조에서도 전처리 깊이 $q = 0 \to 1$ 은 inner iterations 13→7 로 줄였지만 W^{-1} batch 는 17→22 로 늘었다. 반복 수만의 감소는 경제성의 증명이 아니다.

낮은 RHS rank 반례도 있었다. RHS rank 가 1 인 세 eigenvalue-group 문제는 basis 3 으로 출력 WRMS 2.29×10^{-8} 을 얻었지만, 넓은 diagonal spectrum 64 변수 문제는 basis 32 에서도 WRMS 1.482 였다. 이때 full relative residual 은 2.42×10^{-9} 였다. 작아 보이는 residual 도 inverse/output gain 을 거치지 않으면 정확도 보증이 되지 않는다.

8-stage reaction–diffusion RHS 의 Python/ NumPy threading 에서는 stage 당 256 변수가 1/4workers 에서 0.139/0.506 ms, 16,384 변수는 0.522/1.694 ms 였다. 131,072 변수는 1/2/4workers 에서 3.645/3.322/3.396 ms 였지만 paired 시간비는 2workers 약 1.048, 4workers 약 1.017 이며 블록 구간이 1 을 가로질렀다. 이 kernel 실험을 native Rayon 의 crossover 로 옮기지 않았다. Python 탐색 구현의 13 시험은 해당 범위에서 통과했다.

8.6 Causal majorant 와 비용 목적함수

유효 tube 와 inverse·derivative bound 가 주어졌다면 stage 오차는

$$E_i \le a_i + \sum_{j<i} L_{ij} E_j, \qquad E \le (I - L)^{-1} a = \sum_{k=0}^{7} L^k a \tag{8.5}$$

로 전달할 수 있다. 엄격한 하삼각 비음수 L 이므로 전역 $\|L\| < 1$ 이 반드시 필요하지 않지만, bound 를 계산한 radius box 의 closure 는 여전히 필요하다. Endpoint·embedded·dense weights 와 raw/native target 변환 allowance 까지 전달해야 한다. 한 Newton correction 의 출력 투영은 nonlinear remainder 를 누락하므로 일반 error bound 가 아니다.

같은 h 와 출력 계약에서 후보 비용 C_a, 승인비용 C_v, fallback 비용 C_f, 성공확률 p 라면 공유 작업 재사용이 없는 모형에서

$$\mathbb{E}[C] = C_a + C_v + (1-p)C_f, \qquad \mathbb{E}[C] < C_f \iff p > \frac{C_a + C_v}{C_f}. \tag{8.6}$$

서로 다른 h 를 쓰는 후보에는 이 비용을 승인된 시간 전진량으로 나누어야 한다. 이 논리가 이후 cap 반례와 하이브리드 adaptive 비교를 해석하는 공통 기준이 됐다.

9. JVP-only coupled step 의 Rust 구현: PR74

9.1 구현 계약

`crates/vigilode-coupled` 은 별도 `[workspace]` 를 가진 opt-in companion 이다. 자율·항등질량, 공급된 JVP 라는 전제에서 twice-reorthogonalized basis 와 $J_0 V_m$ 을 만들고 작은 전체 stage affine seed 및 Newton/homotopy 보정을 수행한다. 큰 상태 Jacobian 을 조립하지 않지만 $m \times m$, $8m \times 8m$ dense 시스템과 current-stage derivative JVP 는 실제로 계산한다. [R74]

각 stage callback 은 같은 고정 iterate 를 읽고 `ParallelExecution` 의 local Rayon pool 에서 동시 수행된다. 전체 차원의 residual, finite/shape 검사, callback·storage 계획 예산, projected singularity, model epoch 를 검사하며 caller 상태를 바꾸지 않는다. 실패한 병렬 task 도 전부 합류하여 attempted/completed work 를 보존한다. JVP 출력을 NaN 으로 초기화해 일부만 쓰고 성공을 반환한 callback 을 정상으로 보지 않는다.

후보의 `candidate` 는 operational full-space residual 을 통과한 root 일 뿐이고 embedded 판정은 별도다. `coupled_or_sequential` 은 허용된 수치·예산 실패에만 기존 JVP-only GMRES fallback 을 실행한다. 입력·callback·모델 변화 오류에 대해 숨은 retry 로 성공을 만들지 않으며 native-K fallback 과 raw-U 후보의 target ID 및 counter 를 분리한다.

9.2 시험과 벤치마크

Fail-closed scaffold 의 RED 에서는 후보 완성 시험 3 개가 실패하고 거부 대조 2 개가 통과했다. 최종은 계약 5, robustness 12, convergence/accounting 2, 총 19 시험 통과였다. Exact Riccati 문제의 $h = 0.2, 0.1, 0.05, 0.025$ 한 step 오차는 $6.3721 \times 10^{-8}, 1.9240 \times 10^{-9}, 4.2504 \times 10^{-11}, 7.9266 \times 10^{-13}$ 로 감소했다. 네 점의 수렴 대조를 일반적 global 5 차 증명으로 확대하지 않았다.

실측은 16 개 설정·5 arms 의 80 accuracy cells, 800 fresh processes, 3,700 timed attempts, 10 balanced blocks 였다. Few-mode 96·384·12,288, broad-spectrum 64, chain·periodic diffusion 96·384 를 포함했다. Raw-U reference 는 독립 Python 과 별도 Rust recurrence 로 계산했으며 untimed 작은 reference 만 dense matrix 를 만들었다. 출력·embedded-vector 차이 WRMS 가 각각 0.1 이하인 root 를 비용 비교에 허용했다. [R74]

77/80 이 root/reference 조건을 만족했으나 embedded 시간오차까지 통과한 것은 20/80 뿐이었다. 큰 h stress case 를 포함한 고정-step 연구이므로 77 개를 성공한 adaptive trajectory 로 세면 안 된다. 64 개 등록 pair 중 61 개가 완전한 표본을 가졌고 broad-spectrum 의 coupled 3 개는 HOLD 였다.

9.3 실제 병렬 효과와 솔버 우위의 차이

단위는 ms/고정 h 의 complete root attempt 다. Pool setup·참조·JSON·결과 폐기는 제외하고 basis·root·full residual 은 포함했다.

문제	C1	C2	C4	순차 MF	Embedded
Few-mode 96, $h = .001$	0.094713	0.132530	0.104152	0.088063	통과
Few-mode 12,288, $h = .001$	12.846546	6.587397	6.172241	5.532020	통과
Few-mode 12,288, $h = .01$	16.449693	8.695530	7.859994	5.555025	미충족
Chain 384, $h = .01$	1.882242	1.286772	0.984021	6.710258	미충족
Diffusion 96, $h = .001$	1.284279	1.090340	0.972373	3.926156	통과
Diffusion 384, $h = .01$	12.964167	8.638934	6.527793	6.841987	미충족

이 표의 few-mode 는 $\epsilon = 1$ 이다. 큰 few-mode 의 $h = .001$ 에서 C1/C4 paired ratio 는 2.0765, family 구간 [2.0202, 2.1531] 이고 결과 bits 도 같았다. 그러나 기존 순차 MF 는 여전히 더 짧았다. 작은 96 변수에서는 threading 이 손해였다. Diffusion 96 의 순차/C4 는 약 3.9874 이며 embedded 도 통과했지만 baseline 은 기존 generic GMRES 설정이고 raw-U/native-K target 차이가 있어 가장 강한 모든 MF 솔버와 같은 실제오차로 튜닝한 비교가 아니다. [R74]

Chain 384 의 큰 root-latency 비율은 embedded WRMS 약 49.9 인 거절할 timestep 에서 나온 값이었다. Broad-spectrum 에서는 full-space leakage 때문에 세 coupled 경로 모두 거절됐으며 failed C4 는 3.105 ms, 직접 순차 3.301 ms, 후보 후 fallback 은 6.617 ms 였다. 항상 먼저 coupled 를 실행하는 정책은 이 반례에서 약 두 배의 비용을 만들었다.

모든 성공 benchmark 는 직접 $\lambda = 1$ corrector 로 충분했다. Subdivision 의 구현·예산 처리는 시험했지만 어려운 실제 물리계에서 continuation 이 추가 이득을 낸다는 근거는 이 timing 에 없다. 큰 few-mode $h = .01$ 의 실제 작업은 basis 3, reduced 24, Newton 2 회였어도 RHS 33·JVP 83 회였음을 함께 보존했다.

9.4 원자료·리뷰의 범위

847 개 finite complete metric, 33 개 거절 관측, 528 개 work ledger, 770 timing/calibration identity, 30 worker-bit 비교, 61 paired median, 82 개 state hash 를 재계산했다. Closeout 은 112 개 보존 hash 와 분할 reference 하나를 확인했다. 원래 요청한 CodeRabbit CLI 설치는 됐으나 인증이 확인되지 않아 실제 review 는 실행하지 못했고 `issues=null` 이다. PR bot review 를 요청한 것과 반환된 review 가 있다는 것도 구분했다. 같은 작성자의 scalar-fsum 검산을 CodeRabbit 또는 독립인원 심사로 재명명하지 않았다. [R74]

10. Residual-first 와 후보 예산의 경제성: PR75

10.1 수학적 방법이 아니라 필요한 계산의 순서를 바꾸다

Eager corrector 는 $H(z_k)$ 와 $H_z(z_k)$ 를 함께 만든 뒤 수렴 여부를 확인했다. 마지막 iterate 가 이미 수렴했거나 update budget 을 소진했어도 쓰지 않을 $8m$ 개의 stage JVP 를 계산했다. 새 `coupled_step_residual_first` 는 같은 stage 상태의 RHS·residual 을 먼저 계산하고, update 가 필요한 경우에만 저장된 동일 상태에서 derivative 를 구성한다. 같은 small solve 와 accumulation order 를 사용하며 full-space 검증은 생략하지 않는다. [R75]

순수·결정론적 callback, 고정 epoch 와 부동소수점 환경, 작업 예산이 다른 지점에서 결속되지 않는 조건이면 accepted iterate 는 귀납적으로 같다. 이는 quasi-Newton 이나 lagged Jacobian 이 아니다. 다만 callback 요청 순서와 실제 derivative batch 수는 바뀌므로 impure callback 이나 매우 작은 batch 예산에 대한 실패 동일성을 무조건 주장하지 않는다.

`coupled_or_sequential_budgeted` 는 후보 JVP cap 을 기존 사용자 cap 과의 최솟값으로 제한한다. 다음 batch 를 통째로 예약할 수 없으면 후보를 거절하고 허용된 보호 fallback 으로 간다. 이 cap 은 projected LU·RHS·fallback 시간을 제한하는 총비용 상한이 아니다.

10.2 실행과 동일성

RED 에서 affine·update-limit 대조의 불필요한 stage JVP 24 회와 비선형 두 update 의 72 대 48 회를 재현했다. 기존 19 시험과 새 14 시험, 총 33 개가 통과했다. 새 시험은 cap clamping, cap0 fallback, phase 별 batch 예산, 실패한 병렬 RHS/derivative 의 합류·계수, epoch 변화, full-space 검증, theta/worker bits 를 포함했다.

10 개 설정·8 arms 의 80 accuracy rows, 8 개 Williams-order blocks, 640 fresh processes, 1,920 timed attempts 였다. 76 개가 root-reference gate, 48 개가 root 와 embedded 를 함께 통과했다. 27 개 성공 pair 의 전체 stage·출력·embedded bits 가 같았고, 18 개 eager/lazy pair 에서는 실제 JVP 감소가 `unused_stage_jvp_avoided` 와 일치했다. [R75]

10.3 시간 감소와 그 한계

아래는 $\epsilon = 1, h = .001$ 의 ms/complete root attempt 다. 다른 캠페인의 host 시간과 합치지 않았다.

문제	Eager1	Lazy1	Eager4	Lazy4	Embedded
Few-mode 96	0.144918	0.119875	0.166981	0.153661	통과
Few-mode 384	0.495930	0.394383	0.352815	0.341368	통과
Few-mode 12,288	18.789624	13.286156	8.997115	7.737604	통과
Chain 384	2.208214	1.519959	1.157966	0.993368	통과
Diffusion 96	2.109009	1.595137	1.506539	1.411171	통과
Diffusion 384	7.081587	4.818066	3.672809	3.119438	미충족

큰 few-mode 의 paired eager/lazy 는 1worker 에서 1.4117, family 구간 [1.3969, 1.4241], 4workers 에서 1.1656, [1.1187, 1.1921] 이었다. 시간 중앙값으로 약 29%, 14% 감소였으나 작은 96 변수 4worker 구간 [0.8788, 1.1770] 은 1 을 포함했다. 그 경우 Lazy1 이 Lazy4 보다 짧았다. [R75]

실제 전체 JVP 는 few-mode 59→35, chain 144→80, diffusion 의 basis 8→12 확장 사례 348→188 로 줄었으며 RHS 는 각각 25·25·49 회로 유지됐다. Broad-spectrum 거절 후보는 704→64 로 줄었지만 여전히 성공하지 못했다. 별도 derivative dispatch 와 state lifetime 비용이 추가되므로 JVP 절감률이 wall-time·peak memory 절감률은 아니다.

같은 host 에서 큰 few-mode 의 generic 순차 MF 는 7.709782 ms, Lazy4 는 7.737604 ms 였다. 이 작은 차이로 보편적인 승자를 정하지 않았다. 최적화를 여러 번 적용했다는 사실만으로 다른 알고리즘보다 우월해지는 것은 아니다.

10.4 고정 JVP cap 은 왜 최적 router 가 아니었는가

문제	Uncapped transaction	cap64	cap256
Chain 384, $h = .001$	1.002387	10.754173	0.995802
Diffusion 96, $h = .001$	1.425858	6.181441	1.422187
Broad-spectrum 64, $h = .02$	8.861882	8.831947	8.870420

Chain 의 유효한 후보는 총 80JVP 였지만 cap64 는 basis 8 회 후 다음 64-vector batch 를 예약하지 못해 저렴한 후보를 중단하고 비싼 fallback 으로 갔다. Diffusion 에서도 비슷한 일이 있었다. 반대로 affine broad-spectrum 에서는 residual-first 가 쓰지 않는 derivative 를 제거한 뒤 basis 24+ 검증 40=64JVP 만 남아 cap64 도 조기 차단하지 못했다. 작은 projected 시스템의 반복 구성·분해 비용은 JVP cap 으로 제한되지 않기 때문이다. 직접 순차 MF 는 4.874071 ms 였으므로 후보 후 fallback 손해가 남았다. [R75]

Cap64/256 은 사전 지정 sensitivity 사례로 남겼으며 결과에 맞춰 재튜닝하거나 기본 최적 정책으로 승격하지 않았다. 자원제약 API 의 명세 준수와 경제성은 다른 성공조건이다. `derivative_batches`, `residual_only_stops`, `unused_stage_jvp_avoided` 가 추가됐고 실제 task 수는 $8(\text{stage batches} + \text{derivative batches} + \text{validation batches})$ 다. 마지막 값은 수행한 일이 아니라 생략한 eager work 이므로 실제 비용에 더하지 않는다.

10.5 검산과 참조 파일 복구

684 개 완료 metric, 630 개 ledger, 270 cap 검사, 608 timing identity, 27 full-stage parity, 18 saved-work 대조, 50 paired median 과 71 개 state object 를 확인했다. 50 비교 중 3 개는 failed-attempt latency 이며 solver speedup 이 아니다. [R75]

Frozen closeout 에서 16,024,519 바이트의 `REFERENCES.json` 이 Git 에 없음을 발견했다. Python 의 분할 기준은 16 MiB 이상이었지만 `find -size -16M` 는 크기를 MiB 단위로 올림해 15–16 MiB 구간 파일을 제외했다. 동일 측정 소스에서 deterministic reference 만 재생성한 SHA-256 이 원래 기록된 `93d84040...fc63091` 과 정확히 같았고, 101 개 archive hash 와 저장 audit 를 재현했다. 후보 solve 와 timing sample 은 다시 만들지 않았다. 이는 수치 실패가 아닌 packaging 실패의 복구다. [R75A]

11. 최신 경로의 동일 호스트 중간점검: PR76

11.1 비교 범위와 실제 gate

기존 native DVODE/Diffsol 비교기를 고정하고 최신 residual-first companion 을 추가했다. Legacy 다섯 문제에는 이전에 정확도로 선택한 solver 별 rtol 을 사용했으며 새 few-mode-96 만 같은 방정식으로 추가했다. 39 개 preflight, 312 timing cells, 8 블록·프로세스당 warmup 1 회와 측정 5 회를 보존했다. Full-state 312 개와 paired median 30 개를 재계산했으며 26 개는 당시 구현의 matched-achieved, 4 개는 diagnostic-only 였다. [R76]

이번 통합 편집에서 명시한 계약 차이. 보고서의 “공통 anchor” 표현과 달리 실제 `measure.py` 는 $\max E_{\text{up}} \le 1.1$ target 및 관측 error ratio ≤ 2 를 사용한다. 따라서 PR71 의 식 (6.1) 과 정확히 같은 gate 가 아니다.

26 개라는 원기록의 분류는 그 구현에 대한 값으로 보존하고, 본 보고서에서 1.0 anchor 기준으로 재분류한 것처럼 쓰지 않는다. 이는 새로운 solver 계산 결과가 아니라 고정 코드의 판정식에 관한 명료화다. [C76]

11.2 Legacy endpoint 재측정

단위는 ms/구간 적분이다. [R76]

문제	Fast	Banded	Diffsol dense	DVODE
Robertson	0.02908	해당 없음	0.08920	0.07789
HIRES	0.22672	해당 없음	0.45563	0.37634
van der Pol	0.20282	해당 없음	0.80924	0.59148
Brusselator 100	2.66248	1.53083	5.30989	0.69930
Brusselator 400	29.86867	6.02362	193.34939	2.63843

작은 stiff 계에서 fast, 공간 이산화에서 DVODE banded 가 유리하다는 이전의 조건부 관측은 유지됐다. 그러나 과거 CI 의 절대시간과 나누어 버전 speedup 으로 만들지 않는다. Diffsol 은 여전히 dense FaerLU 구성뿐이다.

11.3 새 few-mode 직접 대조

문제는 $\dot{y}_i = a_i y_i + y_i^2$, $a_i = (-1, -20, -100)$ 반복, $y_i(0) = 0.1$, $N = 96$, $t_f = .001$ 이다. 입력 rtol 은 모두 10^{-6}, reference 는 Riccati 닫힌해였다.

경로	시간 ms	실제 E_{up}
Coupled eager	0.103817	9.704179×10^{-13}
Coupled lazy	0.086967	9.704179×10^{-13}
VigilODE fast	0.023882	9.704179×10^{-13}
Diffsol BDF	1.054369	1.063559×10^{-8}
DVODE	0.215195	6.818933×10^{-7}

Eager/lazy paired ratio 는 1.1969 였고 같은 오차 수준의 fast 는 lazy 보다 중앙시간 기준 약 3.64 배 짧았다. 외부 BDF 의 오차는 크게 달랐으므로 그 시간비는 동일 정확도 순위로 채택하지 않았다. 고정 한 coupled step 과 BDF 의 기동·적응 구간이라는 차이도 있다. 이 결과는 작은 저비용 문제에서 coupled homotopy 를 무조건 기본으로 쓰는 정책을 지지하지 않는다.

첫 실행의 source/receipt commit 혼동, 다음 실행의 "정확히 30 pair" 를 `>30` 으로 검사한 auditor 오류는 별도 실패로 보존했다. 마지막 통과가 과거 실패를 지우지 않는다. 특히 paired 관측 범위는 bootstrap confidence interval 과 같지 않으므로 섞어 표현하지 않는다.

12. JVP 를 사용하지 않는 물리·대수적 경로의 발산 탐색

12.1 계약을 강화하되 기존 구조를 버리지 않다

이 탐색에서는 assembly-free JVP 와 더 강한 **runtime nonlinear-derivative-oracle-free** 를 구분했다. 첫째는 전체 Jacobian 을 저장하지 않지만 supplied JVP 를 쓴다. 둘째 범주인 FD-Jv·complex-step·AD 도 상태 미분정보를 쓰므로 강한 요구의 완성은 아니다. 세 번째는 nonlinear Jacobian·JVP·FD/AD 요청 없이 직접 제공된 물리적 flux 계수, 반응식, 보존식, polynomial/rational graph 와 domain 증명을 사용한다. 알려진 선형 수송의 적용까지 금지하는 의미는 아니다. [A4; A6, §3]

기존 RODAS5P target 의 W 에서 J 를 제거하거나 임의 frozen coefficient 로 바꾸면 일반적으로 다른 target·방법이다. 같은 이름이나 차수 보장을 자동 상속하지 않는다. 새 경로는 원래 integral equation 과

실제 곡선의 continuous defect 로 정당화하고, 기존 supplied-JVP RODAS5P 는 별도의 보호 후보로 유지하는 것이 제안의 핵심이었다.

사용자의 고유 연구에서 가져온 구성요소는 promotion theory 의 소멸다항식·finite jet, Loop12 의 모든 초기모드를 보존하는 collective/transverse 및 Volterra 축약, certificate atlas 의 Bernstein·SOS·parameter cell, PR70 chart/cofactor 와 off-manifold forcing, packet trajectory 의 상속 반경·rollback, PR101 의 봉인된 model/provider 다. 이들을 연결한 것은 새로운 구성적 확장이며, 기존 일반 packet 의 `roundoff_enclosed=false` 나 chart 가 없는 generic RVJ 전이의 실패를 지워 모든 경우의 인증으로 승격한 것이 아니다. [A6, §2]

12.2 정확한 collective/Volterra 축약

고정 mesh 의 cell-integrated 복사 상태 q, 작은 물질 상태 z 에 대해 모델이 정확히

$$\dot q = Lq + UF(V^T q, z) + b(t), \qquad \dot z = G(z, V^T q) \tag{12.1}$$

형태라고 하자. L 은 모델에서 주어진 선형 수송·산란 연산자이고 시간 역수 단위다. $m = V^T q$, $\Phi(t,s) = e^{(t-s)L}$ 이면

$$\begin{aligned} q(t) &= \Phi(t,t_n)q_n + \int_{t_n}^{t} \Phi(t,s)[UF(m(s),z(s)) + b(s)]\,ds, \\ m(t) &= V^T\Phi(t,t_n)q_n + \int_{t_n}^{t} V^T\Phi(t,s)[UF(m(s),z(s)) + b(s)]\,ds. \end{aligned} \tag{12.2}$$

이는 변분상수법의 정확한 소거다. 비선형 계산은 작은 memory 에 놓이지만 전체 초기 q_n 는 자유응답과 복원에 남는다. 일반적으로 m 은 Markov ODE 가 아니며 $V^T L = A_r V^T$ 같은 추가 closure 조건이 있어야 선형 부분이 관측량에 닫힌다. 표준 항등식 자체의 신규성을 주장하는 대신, 기존 packet·오차·비용 계약을 물리적 band L 로 확장한 설계라는 범위를 유지한다.

주기 stencil 의 제조 모형

$$\dot q = \nu L_\Delta q - \lambda q - \alpha \bar q^2 \mathbf{1}, \quad \nu = .01,\ \lambda = .7,\ \alpha = .4$$

에서 평균은 $\dot{\bar q} = -\lambda \bar q - \alpha \bar q^2$ 로 닫히고 transverse 는 Fourier flow 다. $N = 32, 128, 1600$, $t \in [0, .3]$, 출력 51 점에서 모든 초기 cell 을 유지했다. 1,600 변수의 stencil spectral radius 는 102400.7 이지만 비선형 차원은 1 이고 nonlinear J/JVP 요청은 0 이었다. 평균 항등식 오차는 1.11×10^{-16}, $N = 32$ 의 독립 tight DOP853 와 차이는 1.78835×10^{-12} 였다. 큰 N 에서 generic solver 와의 시간 비교나 비균일 phase-space FVM 의 성공은 주장하지 않았다. [A6, §4]

12.3 고랭크 흡수에는 조건부 waveform 이 필요하다

$-\kappa(z) D_\sigma q$ 는 물질변수 z 의 수가 작아도 q 에 대한 rank 가 N 일 수 있다. $\sigma = (1,3)$, $q = (1,0)$, $p = (0,1)$ 은 평균이 모두 0.5 지만 평균 변화율은 각각 $-0.5, -1.5$ 다. 단순 mean closure 는 이 둘을 구분하지 못한다.

제안한 일반화는 trial 물질 waveform $z(\cdot)$ 에 조건부인 복사 propagator 를 구하고

$$q[z](t) = \Phi_z(t,t_n)q_n + \int_{t_n}^{t} \Phi_z(t,s) b[z](s)\,ds,$$

$$\mathcal{S}[z](t) = z(t) - z_n - \int_{t_n}^{t} G(z(s), V^T q[z](s))\,ds = 0 \tag{12.3}$$

를 작은 물질·면 waveform 에서 푸는 비선형 Schur memory 다. 이는 물질을 시간 동안 영구 동결하는 것이 아니다. Trial 을 갱신하고 마지막에 원래 결합식의 defect 를 검사한다. 물질이 공간적으로 많으면 작은 차원 가정도 다시 평가해야 한다. 서로 다른 시각의 $L[z]$ 가 commute 하지 않으면 Φ_z 를 단순 exp $\int L\,dt$ 로 대체하지 않는다. 이 일반 nonlinear waveform 경로는 설계 후보이며 PR77 에서 전체가 구현된 것은 아니다. [A6, §4]

12.4 국소 반응 ideal 의 대수적 소거

같은 반응량 단위로 정규화한 toy 반응

$$\dot{u} = -ku(1-x), \qquad \dot{x} = ku(1-x), \qquad u + x = Q$$

에서 $a = hk$ 인 implicit local balance 는

$$x - x_0 - a(Q-x)(1-x) = 0, \quad ax^2 - [1 + a(Q+1)]x + x_0 + aQ = 0. \tag{12.4}$$

이는 ideal $\langle u + x - Q,\ x - x_0 - au(1-x)\rangle$ 에서 u 를 제거한 식이다. $u_0 \geq 0$, $0 \leq x_0 \leq 1$, $a \geq 0$ 이면

$$D = 1 + 2a(Q + 1 - 2x_0) + a^2(Q-1)^2 \geq 1,$$

$$x_+ = \frac{2(x_0 + aQ)}{1 + a(Q+1) + \sqrt{D}}, \qquad u_+ = Q - x_+. \tag{12.5}$$

물리구간 $[x_0, \min(Q,1)]$ 에서 $g(x) - g(y) = (x-y)[1 + a(Q+1-x-y)]$ 의 부호와 endpoint 부호를 이용해 root 의 존재·유일성을 확인한다. 작은 root 를 큰 수의 차로 계산하지 않는 형식이다. 네 초기조건과 여섯 a 의 24 검산에서 보존·물리구간을 유지했고 정규화 balance residual 의 최대는 약 6.99×10^{-12} 였다. [A6, §5]

이 map 은 **backward-Euler resolvent** 다. 그 자체가 5 차 적분기는 아니다. 고차 waveform/collocation 의 nonlinear preconditioner·predictor 로 쓰거나 별도 고차 결합을 검증해야 한다. Elimination·resultant·SOS 는 국소 반응·작은 물질 block·면 연결에 적용하는 후보이지 전체 1,600 변수에 거대한 Gröbner 계산을 강제하는 제안이 아니다. Rational denominator 를 제거해 생기는 가짜 root 는 denominator 와 physical inequalities 로 배제해야 한다.

12.5 Band–Schur, quotient action, causal scan

물리적 선형 balance $W = \left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right)$, $S = D - CA^{-1}B$, $P = \left(\begin{smallmatrix} A & 0 \\ C & S \end{smallmatrix}\right)$ 라면

$$P^{-1}W = \begin{pmatrix} I & A^{-1}B \\ 0 & I \end{pmatrix}, \qquad (P^{-1}W - I)^2 = 0. \tag{12.6}$$

정확한 연산에서는 GMRES 가 최대 2 회에 닫힌다. Band 차원 32·128, border 차원 2 의 실제 대조에서 diagonal 전처리는 34·130 회, exact Schur 는 2·2 회였고 후자의 상대 잔차는 $3.13\times10^{-15}, 7.98\times10^{-15}$ 였다. 근사 Schur·부정확한 band solve·바뀌는 전처리기에서는 nilpotence 가 깨지므로 “항상 3–5 회” 를 보장하지 않는다. 직접 elimination 도 가능하여 이 iteration 수만으로 최적 시간 우위를 주장하지 않았다. [A6, §6]

정확한 tensor $L = L_E \otimes I + I \otimes L_\theta$ 와 commutation 이 있으면 분리된 작용을 사용할 수 있다. 방향성 balance $x_i = \alpha_i x_{i-1} + \beta_i$ 는 map 합성 $(\alpha_2\alpha_1, \beta_2 + \alpha_2\beta_1)$ 로 tree scan 을 구성할 수 있다. 그러나 실수 연산의 associativity 와 부동소수점 bits 는 다르므로 rounding radius 를 전달해야 한다. 소멸관

계 $p(L)U = 0$ 가 인증되면 forced action 의 quotient 를 줄일 수 있지만 $p(L)q_0$ 가 작지 않으면 자유 초기응답까지 줄일 수 없다. Generic band diffusion 의 최소다항식 차수는 커질 수 있다.

여기서 causal 은 단순히 공간 삼각행렬이라는 뜻이 아니다. 주기·양방향 산란은 strongly connected block 으로 남겨야 하고, accepted time-window 의 초기값 의존성은 유지된다. 공간적으로 비국소적인 implicit diffusion 이 시간 인과성을 깨지는 않는다. 미래 참해·사후 reference 를 runtime 승인이나 선택에 사용하지 않는 것도 계약의 일부다.

13. 연속 defect 로 root·시간·출력 오차를 연결하다

13.1 후보와 인증의 분리

실제 반환 곡선 $\hat{y}(t)$ 에 대해

$$d(t) = \dot{\hat{y}}(t) - F(t, \hat{y}(t)) \tag{13.1}$$

를 원래 모델에서 계산한다. 생성형 polynomial/rational graph 라면 합성·Bernstein 계수·denominator 검증으로 시간창 전체를 감쌀 수 있다. 몇 개 출력점의 비교로 $\sup_t \|d(t)\|$ 를 인증하지 않는다. 다항식의 시간미분은 상태 Jacobian/JVP oracle 과 구분된다. [A6, §§7–9]

고정된 단위 일관 metric M 와 유효 tube 에서

$$\langle x - y, F(t,x) - F(t,y) \rangle_M \le \mu(t) \|x - y\|_M^2$$

를 exact identity 또는 semialgebraic 부등식으로 보이면,

$$r(t) \le e^{\int_{t_n}^{t} \mu(s)ds} r_n + \int_{t_n}^{t} e^{\int_s^t \mu(v)dv} \delta(s) ds, \quad \|d(s)\|_M \le \delta(s). \tag{13.2}$$

이 관계는 runtime Jacobian norm 계산 없이도 상속 오차를 전달할 수 있다. Incidence B, 양의 conductance K, cell measure M 의 diffusion $F_{\text{diff}}(y) = -M^{-1}BKB^T y$ 에는

$$e^T M[F_{\text{diff}}(x) - F_{\text{diff}}(y)] = -(B^T e)^T K (B^T e) \le 0 \tag{13.3}$$

가 성립한다. 큰 음의 diffusion eigenvalue 를 절댓값 norm 의 양수로 바꾸어 무익한 $e^{h\|L\|}$ 증폭을 만드는 것을 피한다. 전체 radiation–reaction 의 같은 metric contractivity 는 별도로 필요하며 positivity·conservation 만으로 임의 norm 의 수축을 추정하지 않는다.

13.2 실제 미분 없는 adaptive scalar 인증

$y' = -2y - y^2$, $y(0) = 1/2$, $t \in [0,1]$ 에서 차수 5 의 다항식 후보를 exact rational 합성 recurrence

$$a_{k+1} = \frac{-2a_k - \sum_{j=0}^{k} a_j a_{k-j}}{k+1} \tag{13.4}$$

로 구성했다. Defect 는 차수 10 이며 Bernstein 계수의 절댓값 최대가 전창 상계다. 두 비음수 상태에 대해

$$(u - v)[F(u) - F(v)] = -(2 + u + v)(u - v)^2 \le -2(u - v)^2$$

와 비음수 불변성으로

$$r(t) \le e^{-2(t-t_n)} r_n + frac12\delta(1 - e^{-2(t-t_n)}) \tag{13.5}$$

를 얻었다. $h \le 1/4$ 의 지수함수는 exact rational alternating Taylor 로 감싸고, polynomial positivity 와 상속 반경을 exact arithmetic 으로 검사했다. 반환 binary64 중심의 exact displacement 및 전창 conversion allowance 도 radius 에 포함했다. [A6, §8]

항목	기록된 결과
Accepted windows	28
Rejected trials	5
Dense-output 추가 query 검사	101
Nonlinear 상태 Jacobian/JVP/FD/AD 요청	0
전체 absolute tolerance	10^{-6}
최대 반환 반경	$8.9333706310 \times 10^{-7}$
90 자리 참조와 최대 관측오차	$1.5435012477 \times 10^{-7}$

Rejected trial 은 accepted time·center·radius 를 바꾸지 않았고 다음 창의 초기반경을 0 으로 재설정하지 않았다. 별도 rational checker 가 28 창·33trial·101 출력 proof data 를 읽었다. Query 표본은 부가 검산이고 연속 인증의 근거는 Bernstein bound 와 비교부등식이다. 이 결과는 해당 scalar 모델·유리수 연산·반환 변환에 한정된다. 일반 FVM 의 인증, production packet 승급, formal proof assistant 또는 속도·zero-allocation 의 증거가 아니다.

13.3 오차 예산의 중복 계산 방지

동일한 실제 곡선의 원래 ODE 에 대한 full continuous defect 를 인증하면 불완전한 endpoint root, 시간 절단, curve projection 의 효과가 이미 그 defect 에 들어갈 수 있다. 같은 효과를 embedded error 나 stage residual 이라는 다른 이름으로 무조건 더하면 중복 부과다. 반대로 근사 opacity·frozen limiter·표에서 계산한 defect 라면 원래 모델과의 차이, 계수 불확실성, rounding 을 비중복 항으로 추가해야 한다.

Semidiscrete ODE certificate 는 원 PDE 의 공간절단오차를 포함하지 않는다. 에너지·이온화율·온도처럼 단위가 다른 상태를 raw Euclidean norm 에 섞지 않으며, ALE 로 metric 이 시간에 따라 바뀌면 그 미분에 따른 비교항도 들어가야 한다.

14. 격자·AMR·ALE 조언에 대한 구성적 교정

14.1 Limiter branch 와 계수 동결

Tube 전체에서 같은 limiter branch 가 선택됨을 증명하고 live state 로 그 branch 수식을 평가하는 것은 원래 함수를 계산하는 일이다. 그러나 slope/weight 의 숫자를 step 시작값으로 고정하는 것은 일반적으로 다른 방정식이다.

반례 $y' = y^2, y_0 = 1$ 에서 한 factor 를 y_n 으로 동결한 부분문제를 정확히 풀면 $y_{n+1} = y_n e^{h y_n}$ 이다. 첫 step 에서 원해와의 차이는

$$e^h - \frac{1}{1-h} = -\frac{1}{2}h^2 - \frac{5}{6}h^3 + O(h^4). \tag{14.1}$$

$t = .4$ 에서 10–320 steps 대조의 global order 는 약 0.954→0.997 로 1 차에 접근했다. Frozen 문제를 정확히 푼다고 원 문제의 5 차가 유지되지 않는다. Branch-crossing 시 창 분할, piecewise interval, 원래 flux defect 또는 live branch 갱신이 필요하다. Smooth rational limiter 도 원래 target·양성·TVD·regularization 단위를 별도로 확인해야 한다. [A6, §10]

14.2 Conservative remap 은 고차 정확도의 충분조건이 아니다

$1+0.2\sin(2\pi x)$ 의 cell content 를 정확히 적분하고 이웃 cell 을 합친 뒤 같은 평균으로 나누는 대조는 총량을 보존했다. 그러나 $N=32,64,128,256$ 에서 합산 절대 content error 는 0.0125403, 0.0062550, 0.00312563, 0.00156258 로 약 1 차였다.

Old/new 공간의 선형 projection P 에 대해

$$\hat{q}^{+}=P\hat{q}^{-},\qquad r^{+}\le|P|r^{-}+\eta_{\mathrm{map}}+\eta_{\mathrm{round}} \tag{14.2}$$

를 전달해야 한다. $P\ge 0$ 이고 각 column 합이 1 이면 content ℓ^1 에서 비증폭이지만 signed 고차 projection 에는 자동 적용되지 않는다. 물질 피드백 moment 의 변경 $V_+^TP\hat{q}^{-}-V_-^T\hat{q}^{-}$ 도 검사해야 한다. 광자 수 보존과 에너지·가열률 보존은 같지 않다. 같은 연산자 closure 를 유지하려면 L_+P-PL_- 의 commutation defect 또는 동등한 공간 consistency 항도 필요하다. [A6, §10]

14.3 One-step, BDF, ALE, zero-allocation 의 실제 범위

CVODE reinitialization 이 history 를 지우고 1 차에서 시작한다는 공식 설명은 확인했다. 그러나 BDF 가 항상 고정된 과거 다섯 step 을 필요로 하거나, remesh 후 반드시 극소 step 에 영구히 갇힌다는 결론은 지지되지 않는다. One-step 방법은 history 이식 부담을 줄이지만 remap 오차·새 domain·stiff order reduction·limiter 변화 뒤의 다음 h 와 전체 차수는 별도 문제다. 같은 물리오차·remesh schedule 로 비교해야 하며 BDF 에 불필요한 restart 를 강요하지 않는다. [A6; E3]

Moving mesh 의 cell content $Q_i=V_i\bar{f}_i$ 에는

$$\frac{d}{dt}(V_i\bar{f}_i)=-\sum_{f\in\partial i}A_f(\mathcal{F}_f-\bar{f}_f v_{g,f})\cdot n_f+V_iS_i$$

와 일관된 geometric conservation law

$$\frac{dV_i}{dt}=\sum_{f\in\partial i}A_f v_{g,f}\cdot n_f \tag{14.3}$$

가 필요하다. 고정 N 만으로 이 의무가 사라지지 않는다. $E^2dE\,d\Omega$ 같은 phase-space measure 에서는 cell volume 과 flux 및 좌표 Jacobian 에 가중치를 일관되게 반영해야 한다. 공변 좌표가 적색편이 drift 를 없애더라도 threshold 와 source 의 시간 의존성은 남는다.

Band LU 는 일반적으로 $O(Nb^2)$ work 와 $O(Nb)$ storage 이며 half-bandwidth b 가 고정일 때만 N 에 선형이다. (E,θ) 배치에서 b 가 angular block 과 함께 커질 수 있고, 산란이 항상 인접 각도만 연결하는 것도 아니다. 상태 벡터 $N_{\max}$ 를 한 번 할당했다고 전체 경로가 zero-allocation 은 아니다. Krylov basis, block RHS, fill/symbolic state, degree/rank, thread scratch, remap buffer, certificate·출력 보존까지 계측해야 한다. 당시 사용한 RODAS5P 는 8-stage 이며 조언의 6-stage 표기와 구분한다. 이 설계 사항은 연구된 조건과 반례이며 완성된 AMR/ALE 구현 결과는 아니다.

15. Exact 구조 축약·국소 AD·연속 trajectory 의 결합: PR77

15.1 강한 미분 금지에서 조건부 하이브리드로 수렴

후속 요청은 모든 미분을 금지하기보다, 구조적으로 제거할 수 있는 영역에서는 기존 대수적 최적화를 최대한 사용하고 병목에서는 전역 Jacobian 을 조립하지 않으면서 AD 등을 허용하는 방향이었다. `crates/vigilode-hybrid` 는 exact collective closure 의 packet, nonlinear cell graph 의 local AD, 정확한 tridiagonal+rank-one shifted solve, 기존 coupled 후보를 같은 trajectory/certificate driver 에 연결했다. 현재 `auto` 는 정확한 algebraic capability 분기이며 비용학습 기반의 범용 최적 router 가 아니다. [A5, §2]

실제 측정 소스는 `49ca3150c68d527a275c855ecf814122ff9c533a`, 실행은 `39378576576` 이다. 당시 게시 HEAD `f738fb48...` 는 결과 파일을 추가한 후속 commit 이었다. 복구 보고서 작성 시 새 추가 문서는 미게시였으나 이후 원격 복구·누적 아카이브에 게시됐다. 본 보고서는 그 편집·보존 이력을 새 solver 실행과 분리한다.

15.2 생성형 반사 격자 모델

정규화한 상태와 시간에 대해

$$\dot{q} = d\Delta_{\text{path}} q - \lambda q + \alpha m(1-q) - \epsilon q^2, \quad m = \frac{1}{N}\sum_i q_i \tag{15.1}$$

를 사용했다. Δ_{path} 는 반사 경계 nearest-neighbor path Laplacian 으로 행·열 합이 0 이다. $d \geq 0$, $\lambda \geq \alpha \geq 0$, $\epsilon \geq 0$, initial state 와 불확실성이 $[0,1]^N$ 에 포함됨을 요구한다. $q_i = 0$ 에서 유입은 비음수, $q_i = 1$ 에서 유출은 비양수이므로 상자 불변성을 얻는다. Model field 는 private 이며 ID 에는 차원과 계수 bits 가 들어간다. [A5, §3]

비교의 계수는 $d = .01N^2$, $\lambda = 2$, $\alpha = .4$, $\epsilon \in \{0, 1\}$ 이었다. 물리 rate 는 이 정규화 시간의 역수 단위다. 실제 광자 에너지·온도·다종 이온화의 서로 다른 단위를 raw norm 으로 합친 모형은 아니다. 이 반사 path 의 even-extension FFT 는 앞 절의 주기 scalar-feedback Fourier 예제와 다른 경계조건·모델이다.

15.3 평균·분산 closure 와 packet

평균식은

$$\dot{m} = (\alpha - \lambda)m - (\alpha + \epsilon)m^2 - \epsilon\,\mathrm{Var}(q). \tag{15.2}$$

따라서 $\epsilon \neq 0$ 에서 분산을 지우고 mean 만 적분하면 다른 모델이다. `admits_packet` 은 ϵ 이 **정확히 0** 이고 지원된 격자일 때만 허용한다. $w = q - m\mathbf{1}$ 이면

$$\dot{m} = (\alpha - \lambda)m - \alpha m^2, \qquad \dot{w} = d\Delta_{\text{path}} w - (\lambda + \alpha m)w.$$

$\beta = \alpha - \lambda$, $I_\beta(t) = \mathrm{expm1}(\beta t)/\beta$, $I_0(t) = t$, $D(t) = 1 + \alpha m_0 I_\beta(t)$ 로 두면

$$m(t) = \frac{m_0 e^{\beta t}}{D(t)}, \qquad w(t) = \frac{e^{-\lambda t}}{D(t)} e^{dt\Delta_{\text{path}}} w_0. \tag{15.3}$$

모든 초기 cell 의 정보를 w_0 에 보존한다. Exact 모델 축약이 FFT·지수함수의 부동소수점 후보까지 자동 인증하는 것은 아니고, 최종 실제 곡선의 defect 가 그 차이를 검사한다. Packet endpoint 자체는 nonlinear derivative 를 요청하지 않지만 quintic curve 의 acceleration 에는 대수적 Jf 가 쓰인다. $\epsilon = 0$ auto 에도 action/JVP counter 8 회가 기록되어 있으므로 전체 경로를 "모든 상태 미분 작용 0 회" 로 재명명하지 않는다. [A5, §4]

15.4 국소 AD 와 band-plus-rank-one shifted solve

Nonlinear graph $n_i = \alpha m(1 - q_i) - \epsilon q_i^2$ 를 cell 값과 mean 에 대해 미분하면

$$J = d\Delta_{\text{path}} - \mathrm{diag}(\lambda + \alpha m + 2\epsilon q_i) + uv^T, \quad u_i = \alpha(1 - q_i), \quad v_i = 1/N. \tag{15.4}$$

FullAd 는 알려진 stencil 까지 dual 수로 통과시키고, LocalAd 는 물리적 stencil 과 nonlinear cell graph 를 분리한다. 일부 cell 만 동적으로 택하는 sparse active-AD 를 구현한 것은 아니다.

$s = h\gamma$, $W = I - sJ = T - suv^T$ 에 대해

$$z = T^{-1}su, \qquad x_0 = T^{-1}b, \qquad x = x_0 + z\frac{v^T x_0}{1 - v^T z} \tag{15.5}$$

로 푼다. Pivot·분모·유한성을 확인하고 현재 원래 target residual 을 다시 검사한다. 전역 $N \times N$ Jacobian 은 만들지 않지만 local derivative 계수·band diagonal·rank-one 벡터는 실제로 구성한다. 정확한 계약은 **global-Jacobian-assembly-free structured AD** 다. 이것은 앞 단계의 절대적인 derivative-oracle-free 보다 약하지만 이번 하이브리드 요청과 일치한다. [A5, §5]

기존 복구에서는 Wolfram 의 $N = 3$ 기호 항등식과 32 개 Fraction 모형으로 dual 방향미분·mean/분산·분해·shifted solve 를 확인했고 136 개 row-gain 대조를 했다. 이 보고서 집필 과정에서도 대표 mean·row-sum·분해 항등식을 다시 확인했지만, 그 작은 편집 검산을 Rust 전체 재실행으로 세지 않는다.

15.5 실제 반환 곡선의 연속 인증과 atomic 승인

Endpoint 후보 y_1 로 cubic 또는 quintic Hermite/Bernstein 곡선을 만든다. Quintic 은 양끝의 값·RHS·acceleration 을 사용한다. 저장된 binary64 Bernstein controls 자체를 실제 곡선으로 보고, 그 곡선에 대한 원래 식 (15.1) 의 defect 를 outward 구간연산으로 감싼다. 모델이 이차이므로 degree p 곡선의 defect 는 최대 $2p$ 다. [A5, §6]

상자 안에서는 J 가 Metzler 이고

$$\sum_j J_{ij} = \alpha - \lambda - \alpha(m + q_i) - 2\epsilon q_i \le \alpha - \lambda \le 0. \tag{15.6}$$

따라서 무한대 norm 의 one-sided gain 을 사용해 큰 diffusion rate 를 양의 오차증폭으로 바꾸지 않는다. 안전한 $0 \le a \le \lambda - \alpha$ 와 defect bound δ 로

$$r(t) \le e^{-a(t-t_0)}r_0 + \frac{\delta}{a}(1 - e^{-a(t-t_0)}) \tag{15.7}$$

를 전파하고 $a = 0$ 은 선형 극한을 쓴다. 참해의 invariant box 와 represented curve 의 Bernstein box 포함이 전제다.

Candidate, actual curve, continuous tube/radius, requested output 의 rounding 반경을 모두 통과해야 time·state·inherited radius 를 commit 한다. 실패 trial 은 rollback 되고 비용은 남는다. Query 별 rounding 이 예산을 넘으면 추가 dense query 도 거절될 수 있다. Root·시간·curve-fit 의 동일 효과를 embedded error 라는 이름으로 중복 부과하지 않는다.

이 인증은 **특정 생성형 반사 격자의 반이산 ODE**, initial box, IEEE round-to-nearest·gradual underflow·no-fast-math/FTZ 조건에 한정된다. PDE 공간오차, remap, 일반 callback, 실제 비평형 복사·물질 모형 전체의 인증은 아니다.

15.6 실제 실행과 full-trajectory 성능

기존 실행은 Rust 19 시험, 정확도 보정·대조 102 회 중 경험적 출력 ceiling 84 회 통과, 선택 40 조건, fresh timing processes 160 개와 480 full trajectories, 32 쌍 비교를 보존했다. 22 쌍의 경험적 오차비가 2 이내였으며 같은 인증 계약까지 갖는 것은 그중 18 쌍이었다. Exact-rational replay 는 residual 계수 87,056 개, 연속 segment 203 개, 반환 출력 600 개를 검사했다. [A5, §7]

구간은 $[0, .2]$, 출력 21 점,

$$q_i(0) = .45 + .12\cos\frac{\pi(i + 1/2)}{N} + .03\cos\frac{7\pi(i + 1/2)}{N}$$

였다. Native 경로는 absolute budget 10^{-6} 을 공유했다. 외부 fast/Diffsol/DVODE 는 사전 tolerance ladder 중 auto 의 실제 출력오차에 가장 가까운 유효 설정을 정확도만으로 고르고, 시간은 별도로 측정했다.

다음은 ms/전체 trajectory 이며 4 개 fresh-process blocks, 각 3 회 적분의 per-arm median 요약이다. [A5, §8]

N, ϵ	Eager full-AD	Residual full-AD	Residual local-AD	Structured local-AD	Auto
16, 0	1.07819	1.04142	1.05500	0.85785	0.79861
16, 1	10.61555	9.48662	9.46909	1.02876	1.01906
64, 0	5.15909	4.96874	4.96820	3.22203	3.04952
64, 1	44.76805	39.27822	39.38370	3.93385	3.93018

$N = 64, \epsilon = 1$ 에서 eager/auto 중앙값의 비율은 11.3908, residual-local/auto 는 10.0208 이었다. **같은 새 trajectory/certificate driver 에서 특정 모델의 endpoint 후보를 교체한 비교**이지 과거 프로그램 전체의 보편적인 11 배 가속이 아니다.

Local AD 만의 이득은 확인되지 않았다. 같은 residual full/local-AD 의 paired median 은 0.998649, 관측 범위 [0.991336, 1.008576] 이었다. 구조적 분리가 합리적이어도 이 작은 모형에서 시간 이득이 입증됐다고 할 수 없다. Eager JVP 8,262 회가 residual 5,606 회, structured 50 회로 줄고 bandfactor 5 회가 기록됐지만, 이 counter 하나를 총시간 예측값으로 쓰지도 않는다.

15.7 병렬 homotopy 성과와 구조화된 순차 성과를 섞지 않는다

이번 내부 최속 `structured_endpoint` 에는 여전히 `for i in 0..8` 의 순차 raw-U stage 가 남아 있다. 큰 순차 solve 를 없애려던 coupled 연구와 달리, 이 대조에서는 구조화된 정확 solve 를 공유한 순차 경로가 더 경제적이었다. 이를 “all-at-once homotopy 가 11 배 빨라졌다” 고 설명하면 잘못이다. 현재 `auto` 는 packet 또는 structured 를 exact capability 로 고르는 제한된 분기다. [A5, §8]

반면 더 많은 미분이 전체 일을 줄인 사례도 있었다. $N = 64, \epsilon = 1$ 에서 cubic-auto 는 27 accepted+1 rejected window, 10.41518 ms 였고 quintic-auto 는 5 accepted, 3.93018 ms 였다. 실제 출력오차는 각각 7.43784×10^{-9} 와 7.45625×10^{-9} 로 가까웠으며 같은 인증 계약의 paired 시간비는 2.67224 였다. Acceleration 의 AD/Jf 비용을 더 쓰더라도 짧은 창·반복 인증을 줄여 전체 비용이 낮아진 것이다. $\epsilon = 0$ 의 cubic/quintic 오차비는 약 10–15 배이므로 그 경우의 시간비를 동일 달성 오차로 부르지 않는다.

15.8 외부 솔버와의 비교 및 타이머 경계

N, ϵ	Auto	Diffsol 0.16.1 dense	DVODE dense
16, 0	0.79861	0.08755	0.04589
16, 1	1.01906	0.08047	0.04887
64, 0	3.04952	0.51822	0.41482
64, 1	3.93018	0.54095	0.46358

외부 두 BDF 의 시간이 더 짧았다는 관측은 그대로 남긴다. 그러나 native 는 연속 certificate 와 반환 반경을 계산하므로 계산 의무가 같지 않다. $N = 64, \epsilon = 1$ 의 세부 정확도는 다음과 같다. 여기의 오차는 **최대 출력 절대오차 + 경험적 참조폭**으로, PR71 의 정규화 WRMS 와 동일 지표가 아니다. [A5, §9]

경로	관측오차 + 참조폭	반환 인증 반경	시간 ms
Auto/structured	7.45625×10^{-9}	5.58137×10^{-8}	3.93018
Diffsol dense	4.36323×10^{-9}	제공하지 않음	0.54095
DVODE dense	3.59921×10^{-9}	제공하지 않음	0.46358
VigilODE dense-fast	5.45452×10^{-12}	제공하지 않음	0.96055

Diffsol 과 auto 의 경험적 오차비는 1.70888 이지만 인증 계약은 다르다. DVODE 는 2.07163 으로 factor-2 기준을 약간 넘으며, 기준을 넓혀 통과시키지 않았다. Fast 는 참조 precision 근처까지 과도하게 정확하므로 같은 달성 오차 비교가 아니다. Global rank-one 결합 때문에 DVODE 는 dense 로 구성됐고, Diffsol 도 dense FaerLU 만 비교했다. 다른 sparse 또는 band-plus-low-rank BDF 의 순위를 이 자료로 판정하지 않는다.

후속 복구에서 `run()` 내부의 `serde_json::to_value` 가 full integration·proof·통계 객체를 materialize 하는 비용을 타이머에 포함함을 확인했다. 저장된 "excluding serialization" 은 바깥 문자열 출력으로만 좁게 읽어야 한다. 원래 시간을 소급 보정하지 않았고, 외부와의 차이를 모두 인증 자체의 불가피한 비용이나 순수 endpoint solver 효율로 귀속하지 않았다.

16. 성과의 통합 해석과 미해결 영역

16.1 실제로 얻은 성과

구현 비용의 절감은 여러 독립 층위에서 확인됐다. Fast·banded 는 같은 step 수에서 generic overhead 와 구조를 활용하지 못한 선형대수 비용을 줄였다. GMRES-into 는 사용하지 않는 QR·할당 반복을 줄였다. Exact factor cache 는 같은 shifted matrix 의 재계산만 제거했다. Residual-first 는 수렴한 iterate 의 미분행렬을 만들지 않았다. 이들은 target 자체를 바꾸지 않거나, 명시된 동일성 검사를 통과한 최적화다.

실행 알고리즘의 변화도 구현됐다. Shared JVP basis 와 작은 all-stage root 를 통해 큰 공간의 여덟 순차 solve 를 대체했고, 실제 stage 작업의 병렬성과 worker-count output bits 를 확인했다. 그러나 작은 문제의 scheduling 손해, 넓은 spectrum 의 leakage, 실패 후 fallback 비용과 범용 baseline 에 대한 성능 한계가 함께 나타났다.

모델 구조를 통한 계산의 우회는 별도 기여다. 정확한 collective closure 와 모든 transverse 초기정보, 국소 reaction ideal, physical band·border 및 quotient action 은 단순히 일반 root solver 를 빠르게 만드는 것과 다른 전략이다. Hybrid 의 큰 내부 개선은 이 구조화된 경로에서 나왔다.

정확도 계약의 강화는 초기 경험적 endpoint 검산에서 actual dense-output 와 inherited error·roundoff 를 포함하는 model-scoped continuous certificate 까지 진전됐다. 다만 이는 특정 생성형 ODE 에 대한 성과이며 과거 모든 generic solver 경로를 인증한 것은 아니다.

16.2 채택하지 않는 결론

아래 명제는 전체 연구의 결론으로 채택하지 않는다.

과도한 주장	실제 자료가 허용하는 판단
VigilODE 가 모든 외부 stiff 솔버보다 빠르다	작은 문제 fast 의 강점과 DVODE banded 의 공간 문제 우위가 공존한다.
Thread 수를 늘리면 언제나 유리하다	callback 비용·메모리 대역폭·dispatch 에 따라 손해도 관측됐다.
RHS rank 가 작으면 Krylov 차원이 작다	넓은 spectrum 반례가 있다.
작은 root residual 이면 출력 정확도가 보장된다	inverse/output gain·nonlinear remainder·time error 가 별도다.
JVP 를 줄이면 전체 시간도 같은 비율로 줄어든다	projected LU, RHS, 인증, dispatch, serialization 이 남는다.
Linear 또는 constant-J 면 exact cache 가 항상 유리하다	shift 변화와 no-hit overhead 때문에 성립하지 않는다.
무조건 미분을 없애는 것이 최적이다	quintic 의 추가 acceleration 이 전체 인증 trajectory 비용을 줄였다.
Conservative remap 또는 limiter freezing 이면 5 차를 유지한다	구체적 1 차 반례와 별도 remap·모델 defect 의무가 있다.
PR78 아카이브 완전성이 과거 모든 raw archive 의 무손실을 뜻한다	네 명시 ZIP·72 개 구성원 완전성이다. 과거 다른 캠페인의 누락 한계는 유지된다.

16.3 자동 선택의 올바른 목적함수

선택변수 $a = (\text{방법}, h, \text{mesh}, \text{degree}, \text{rank}, \text{workers}, \text{certificate route})$ 에 대해

$$\mathcal{J}(a) = \frac{\mathbb{E}[C_{\text{setup}} + C_{\text{trial}} + C_{\text{certificate}} + C_{\text{reject}} + C_{\text{fallback}} + C_{\text{remap}}]}{\mathbb{E}[\Delta t_{\text{accepted}}]} \tag{16.1}$$

를 최소화하되 물리 domain·invariant·요구 출력오차를 제약으로 둬야 한다. 사용 가능한 정보는 현재 accepted state 와 허용된 trial/probe 여야 하며 미래 참해나 사후 reference 를 online 판단에 사용하지 않는다. 기존 연구의 영향도 기반 residual 배분은 검증된 contraction·비용 모형 안에서 활용할 수 있지만 모든 미래 입력의 전역 최적성을 뜻하지 않는다. [S05; A6, §11]

현재의 capability selector 는 이 목적함수를 풀었다고 볼 수 없다. Fixed JVP cap 의 실패는 후보 차원, projected solve 비용, leakage 감소, 남은 iteration, proof boundary crossing 과 이미 소비한 비용을 함께 보아야 함을 보여 준다.

16.4 남은 연구의 범위와 최소 종료조건

다음 작업은 새 기능을 모두 동시에 추가하는 대신, 기존 결과에서 확인된 병목을 분리해야 한다. 첫째, PR77 의 endpoint·curve·interval enclosure·proof 객체 생성 비용을 별도로 측정하고 동일한 계산 의무의 baseline 을 확보해야 한다. 둘째, 같은 실제 출력오차에서 full adaptive trajectory 의 비용을 비교하고 root 성공과 timestep 승인을 계속 분리해야 한다. 셋째, 작은 생성형 모형의 구조·증명을 실제 비균일 (E, θ) FVM, 다종 물질, 비국소 opacity·산란으로 옮길 때 성립하는 조건부터 증명해야 한다.

AMR 은 accepted-window 사이의 conservative remap·moment·radius·epoch·commutation defect 를 포함한 작은 검증부터, ALE 는 GCL·시간 의존 metric·좌표 measure 를 분리한 검증부터 시작해야 한다. Zero-allocation 은 capacity 설계가 아니라 allocator 계측으로 확인해야 한다. 전체 workspace·다중 호스트 성능·외부 독립인원 심사도 아직 별도의 의무다. 이 항목들은 본 보고서의 제안이며 이번에 구현·완료된 결과로 표시하지 않는다.

17. 출판·복구·백업과 결과의 재현성

17.1 Source, execution, publication 의 세 정체성

각 실행은 실제 빌드한 source SHA, 원자료 run ID, 이후 결과를 게시한 HEAD 를 분리한다. Source 가 같은 읽기 검산은 새로운 적분이 아니고, packaging 복구는 과거 timing sample 을 다시 만들 수 없다. 새 host 에서 solver 를 재실행하면 재현 실험의 새 표본이지 원래 시간 바이트가 아니다.

PR71 의 주 screening 에서 20 MiB 초과 raw gzip 두 개가 빠졌다는 한계, 초기 V2 evidence ZIP 의 약 184 MB·10,156 파일이라는 원문 기록, PR75 의 deterministic reference exact-byte 복구, PR78 의 네 독립 ZIP 완전성은 **서로 다른 보존 대상**이다. PR78 의 76/76 검증으로 앞의 모든 대형 evidence 가 소급 완전 보존됐다고 주장하지 않는다. 본 통합 작업도 초기 184 MB 전체를 재검증하지 않았다. [A2, 결과 파일; A3, §8; R75A; R78]

17.2 마지막 원격 보존 상태

통합 편집 시 확인한 PR78 은 Draft·미병합이며 HEAD 는 `ebb5b81ebe77314b6fee941096fd6cba6d8597fc` 다. 원격 closeout 은 네 original ZIP 과 그 내부 72 파일을 Git tree 의 blob SHA-1·크기로 대조해 76/76 일치, Drive 의 전체 raw ZIP SHA-256 을 4/4 일치로 기록한다. 이는 증거 장부의 읽기 확인이며 이 보고서 작성 중 모든 Drive 바이트를 다시 다운로드했다고 주장하지 않는다. [R78]

원본 archive	bytes	내부 파일	SHA-256 prefix
Integration research	47,810	22	`54f4f4a99d55`
Derivative-free FVM	121,653	18	`a17864dd8fa1`
Legacy VODE/Diffsol	18,869	11	`790b4621e09d`

원본 archive	bytes	내부 파일	SHA-256 prefix
Hybrid AD recovery	66,374	21	`aea357e42f53`

원본 전체 SHA-256 은 다음과 같으며, prefix 만으로 무결성을 판정하지 않는다.

```
Integration:
54f4f4a99d552388fc28d01f5f367176e301c2af15c3cd8a9c7a30b4e5dfd4f1
Derivative-free FVM:
a17864dd8fa1951c7ddcd236740c564c10b323248b1bb30c92eea32bc1e745e7
Legacy VODE/Diffsol:
790b4621e09dff21e196cc548b7851ca751d09f7b0287c52e0bc857fca2ad9ae
Hybrid AD recovery:
aea357e42f53e027b18ab012a5cb9fb554bd3d0f2ab2a641a7eaa27b6632c656
```

네 ZIP 합계는 254,706 bytes, 압축해제된 72 개 원본 합계는 736,335 bytes 다. 로컬에 제공된 네 ZIP 의 CRC·SHA-256 과 입력문서 hash 는 이번 편집의 검증 파일에 따로 기록했다. 원본 실험결과를 덮어쓰거나 신규 연구 commit 을 원격에 게시하지 않았다.

17.3 대형 환경 자원

Rust 1.94.1 오프라인 배포본 192,287,020bytes 는 Drive 의 part00·01·02 로, RUSTCORE 원본 33,103,685bytes 는 별도 ZIP 으로 보존된 경로가 게시돼 있다. 대형 바이너리 환경 자원과 과학 source/measurement 를 구분한다. 같은 compiler version 설치가 같은 timing 이나 바이너리 bytes 를 보장하지 않는다. Drive 공유 권한은 기존 설정을 유지했으며, 링크를 여는 데 해당 계정 접근권한이 필요하다. 복원 상세는 R78 의 `LARGE_DRIVE_BACKUPS_KO.md` 와 `REPRODUCE_ORIGINAL_ARCHIVES.md` 를 따른다.

18. 최종 결론

이 스레드는 “실행되는 솔버” 를 “빠르고 정확하다고 말할 수 있는 조건이 명확한 연구 코드” 로 만드는 과정이었다. 초기 설치·빌드에서 generic 와 특화 경로의 큰 비용 차이를 확인했고, 조밀 출력·분석기·wrapper 의 오류가 단순 시간표의 해석을 제한한다는 사실을 드러냈다. 실제 달성 오차를 맞춘 외부 비교는 작은 계의 fast 경로와 공간 계의 banded BDF 가 서로 다른 영역에서 강하다는 결론으로 수렴했다.

그 뒤 안전한 exact 재사용과 residual-first 는 같은 결과를 유지하는 비용 절감의 구체적 사례를 만들었다. Coupled-stage homotopy 는 큰 공간의 여덟 순차 solve 를 대체할 수 있음을 실제 Rust 와 병렬 실행으로 보였지만, rank·leakage·동기화·fallback 의 비용을 무시하면 성능이 악화될 수 있음을 함께 확인했다.

미분 없는 탐색은 일반 solver 의 호출 구조를 최적화하는 것에서 나아가, 정확한 물리·대수 구조를 이용해 비선형 문제 자체를 낮은 차원으로 바꾸는 가능성을 제시했다. 그 강한 계약을 모든 모델에 강요하지 않고 필요한 곳에 국소 AD 를 허용한 하이브리드 구현은, 제한된 생성형 모델에서 adaptive trajectory 와 dense-output 의 연속 defect·상속 반경·반환 rounding 을 하나의 승인 체계로 연결했다.

따라서 이 연구의 완결된 결론은 “항상 가장 빠른 새로운 솔버를 얻었다” 가 아니다. **정확히 어떤 계산을 생략할 수 있는지, 어떤 구조가 정확한 축약을 허용하는지, 어떤 결과가 승인 가능한 시간 전진인지, 그리고 그것을 얻는 전체 비용이 얼마인지 구분하는 구현·이론·증거 체계를 얻었다**는 것이다. 이 구분을 유지해야 향후 실제 복사·물질계, 큰 비균일 격자, 자동 선택과 AMR/ALE 연구가 재현 가능한 과학적 진전으로 이어질 수 있다.

부록 A. 고정 소스·실행·게시 장부

아래 표에서 **실측 소스**는 해당 바이너리 또는 실험 코드를 만든 커밋이고, **게시 기준**은 결과·복구 문서를 읽을 수 있는 커밋이다. 같은 날짜나 같은 브랜치라는 이유로 서로 바꾸어 인용하지 않는다. 표에는 가독성을 위해 짧은 prefix 를 쓰고, 정확한 값은 뒤의 source register 와 동봉한 `SOURCE_REGISTER.json` 에 둔다.

연구 단위	실측·구성 소스 prefix	주요 실행 ID 또는 자료	게시 기준 prefix
초기 Rust 설치·첫 benchmark	`ccaeaa50beb4`	154 조건 및 A07, 초기 evidence bundle	PR71 의 원본 CSV 보존
V2 정밀 benchmark	`8a8bd409b1d9`	CT 2,700 조건, allocation 84 조건, Callgrind 14 조건	`a21d573980f6` 는 후속 source-equivalence 확인
PR71 주 screening	`87df9b071c3a`	`39257390707`	`64b9852124b6`
PR71 오차 정합	`9c6a6d914579`	`39261895983`	`64b9852124b6`
PR71 고정 결과 readback	`65ef8cde8b3c`	`39263860973`	`64b9852124b6`
PR73 banded 개선	코드 `b51c051b045e`; 측정 `f8e89b70528d`	`39304607635`; 후속 고정 결과 복구	`3c0157910076`
통합 이론·Python prototype	조사 기준 `c03ca90f19a9`	독립 Integration ZIP	원본 ZIP 은 PR78
PR74 native coupled	`1e4a993874d6`	`39349031422`; closeout `39349959195`	`15c43ce76734`
PR75 residual-first	`f08d152a35fb`	`39355449893`; 참조 복구 `39357596507`	`6c945b6c6e0e`
PR76 중간점검	`aaa8116cbb20`	`39360536475`	`6972f28aa4bc`
미분 없는 FVM 탐색	조사 기준 `c03ca90f19a9`	독립 DerivativeFree ZIP	원본 ZIP 은 PR78
PR77 hybrid trajectory	`49ca3150c68d`	`39378576576`; 후속 recovery 9 시험	`9bb05bed4cd7`
PR78 누적 보존	새 과학 실행 없음	4 ZIP·72 파일·원격 closeout	`ebb5b81ebe77`

초기 V2 의 실측 이후 Rust source 가 동일했던 `a21d573` 과 실제 빌드 `8a8bd40` 의 관계는 **입력 소스 동등성** 이며 바이너리 byte identity 가 아니다. PR76 의 중간점검에는 PR71 의 고정 native comparator 와 후속 coupled companion 이 함께 들어갔으므로, comparator 의 과거 소스만을 전체 최신 바이너리의 정체성으로 사용해서도 안 된다. [A2; A3; R76]

A.1 주요 커밋의 정확한 값

```
initial_measured = ccaeaa50beb4580505a36d25ccb919ec3ca38c7f
v2_measured = 8a8bd409b1d96c7b7421f496d229e119fd3d284e
v2_later_rust_equivalent = a21d573980f66fe02891bd216153443b5352f921
strict_screen_measured = 87df9b071c3a3a7828e999e553a088af5860d274
strict_matched_measured = 9c6a6d9145795e35b978652a567b806ddc84a6bd
strict_readback_source = 65ef8cde8b3c6fdbfef4d06b52c63df673467651
pr71_publication = 64b9852124b67eedbb8575b42c01127e4b86283b
banded_code_patch = b51c051b045e977c038e5eec468de0004840872b
banded_measured = f8e89b70528da8490fbe1febf0b0ede9ff84a01e
pr73_publication = 3c0157910076ce552283c831c2be000c66671fab
integration_theory_base = c03ca90f19a9e64ba7751c069325a48b54b39c2e
coupled_measured = 1e4a993874d6930ea2bb41bb0ff7392f21c33df7
pr74_publication = 15c43ce76734d0a560fcec5909ac49f1e6f2d347
economy_measured = f08d152a35fb2db748692570969cb1723c287c26
pr75_publication = 6c945b6c6e0e024d8f733c21da8d35fbcc3c5f2a
midcheck_measured = aaa8116cbb20842eb5ebe308939670f6dd93bb0a
pr76_publication = 6972f28aa4bcbe4c68dbba92e44ff58884545bca
hybrid_measured = 49ca3150c68d527a275c855ecf814122ff9c533a
hybrid_original_evidence = f738fb481bad3f086b568c8493105576851093f2
pr77_recovery_publication = 9bb05bed4cd7df720f2b429c2b47e97ed76712d5
pr78_archive_closeout = ebb5b81ebe77314b6fee941096fd6cba6d8597fc
```

부록 B. 시험·실패·정정의 통합 장부

B.1 시험 수의 해석

연구 단위	기록된 시험·검산	해석상 제한
초기 설치/benchmark	Rust 40 passed, 2 ignored	8 개 표적 target; 전체 workspace 아님
V2	DP-02 4 passed; triage CLI 21 passed, 4 ignored	Python 90 개 method 의 2 개에서 6 개 subfailure 별도 보존
PR71	native 표적 4 passed; 계약 10 개와 serialization 3 개 실행	최종 초기값 bit FAIL 은 별도 floating-consistency 로 설명
PR73	회귀 9 개, 관련 내부 17 개 통과	전체 workspace 는 660 초 timeout
통합 Python	13 개 통과, 30 개 homotopy 설정	Rust 나 전체 trajectory 검증 아님
PR74	19 개 Rust 시험 통과	후보 root 와 물리 step 의 승인 분리
PR75	33 개 Rust 시험 통과	기존 19 개를 포함; 19+33 을 고유 시험 52 개로 합산하지 않음
미분 없는 scalar	28 창·33trial·101query 의 proof-data 검사	명시한 scalar 모델과 정확 rational 산술에 한정
PR77	Rust 19 개; 87,056 계수·203segment·600 출력 replay	생성형 모델 전용; 독립인원 review 아님
PR77 후속 복구	새 로컬 9 개 시험, CSV·대수식 대조	원래 전체 replay 를 다시 실행한 것으로 세지 않음
PR78	76 개 Git 객체·4 개 Drive ZIP identity	archive completeness, 과학적 승급 아님

이 표의 숫자를 단순 합산한 “총 PASS 수” 는 만들지 않는다. 반복된 이전 시험, 서로 다른 granularity 의 test method·수치 조건·계수·출력·파일 hash 가 섞여 있기 때문이다. [A1–A6; R73–R78]

B.2 중요 실패와 해석 변경

대상	관측 또는 반례	분류와 최종 처리
초기 CLI 링크	CGU=16 에서 undefined hidden symbol	실행환경/빌드 문제; CLI CGU=1 로 실행, 특정 compiler bug 미확정
A07 외부 비교	Diffsol 정상 실행이나 grid WRMS 기준 미충족	수치 정확도 정합 실패; 실행 실패나 일반 solver 결함으로 확대하지 않음
초기 조밀 출력	Enforce 후에도 실제 normalized error 가 1 초과	local estimator 와 출력/전역오차 계약 분리
V2 검사기	길이가 다른 state/time 배열을 허용하면 가짜 zero error	합성 분석기 반례; 실제 solver 가 그 행을 출력했다고 주장하지 않음
SciPy 1.17 VODE	analytical dense Jacobian 전치	원본 Fortran VODE 열세 해석 철회; 원래 측정은 보존
SciPy 전치 workaround 뒤 이상	일부 dense 경로의 과도한 step 잔존	원인 미해결; 두 번째 구체적 LU/pivot bug 로 단정하지 않음
PR71 초기값 bit 검사	Diffsol 18 행, 최대 1 ulp 규모 차이	원래 FAIL 보존, 원자료 유지, floating-consistency 범위만 별도 통과
PR73 banded solve	비유한 입력/해를 성공 처리하거나 size overflow panic	네 회귀를 실제 수정; 작은 유효 pivot 은 허용
PR73 retry 의심	원본 retry 시험 4 개 통과	의심 반증, 불필요한 생산 코드 수정 배제

대상	관측 또는 반례	분류와 최종 처리
Exact LU cache	nonlinear J 또는 변하는 shift 에서 hit 0	기본 활성화 불채택; no-hit 비용 보존
Coupled broad spectrum	작은 projected 공간 밖 leakage	후보 거절; 실패 후 fallback 의 중복비용 보존
고정 JVP cap	좋은 후보를 끊고 작은 LU 비용은 못 막음	안전한 예산 API 이나 최적 경제성 정책은 반증됨
Limiter 계수 동결	$y' = y^2$ 반례에서 global 1 차	고차 보존 주장 불채택
Conservative remap	총량 보존과 동시에 1 차 오차	5 차 복원·물리 moment 보존 주장 불채택
Homotopy/병렬 실측	concurrency 는 존재하나 모든 문제에서 가속은 아님	실행가능성·worker 가속·solver 순위를 구분
PR77 외부 비교	인증 없는 BDF 가 더 짧음	외부 우위 결과를 보존; 인증 의무·실제 오차·타이머 차이 명시
CodeRabbit	CLI 설치 후 인증 확인 실패	리뷰 미실행, `issues=null`; 수치 audit 로 대체 표기하지 않음
게시 크기 필터	`find -size -16M` 반올림과 Python 임계값 불일치	exact reference 복구와 byte 단위 게시 필터; 원래 실패 기록 유지

부록 C. 재현 절차와 원자료 탐색

C.1 먼저 재현의 목적을 선택한다

고정 원자료 읽기 검산은 과거 상태·참조·시간 표본을 읽어 같은 지표와 집계를 얻는 작업이다. 새 integration 을 하지 않는다. **소스 고정 새 실험**은 같은 코드와 설정으로 새 host 에서 실행하여 새로운 시간 표본을 얻는 작업이다. **원본 archive 복원**은 바이트·파일 구조의 보존을 확인하는 작업이다. 어느 하나가 나머지를 자동으로 보장하지 않는다.

각 연구 노드의 `REPRODUCE.md` 와 실제 workflow 에 기록된 명령을 우선한다. 아래는 실행 경로를 찾기 위한 최소 진입점이며, 이 통합 집필에서 재실행한 명령이라는 뜻은 아니다.

```
# 기존 저장소의 깨끗한 상태에서, 별도 worktree로 native coupled 재현
# 측정 source를 사용하므로 이후 문서 HEAD를 측정 바이너리로 오인하지 않는다.
git worktree add --detach /tmp/vigilode-coupled-replay \
  1e4a993874d6930ea2bb41bb0ff7392f21c33df7
cd /tmp/vigilode-coupled-replay
cargo +1.94.1 test \
  --manifest-path crates/vigilode-coupled/Cargo.toml --locked
cargo +1.94.1 build \
  --manifest-path crates/vigilode-coupled/Cargo.toml \
  --profile measurement --example coupled_bench --locked
```

Residual-first 는 `f08d152a35fb2db748692570969cb1723c287c26` 의 `economy_bench` 와 해당 노드의 benchmark/audit 를 사용한다. Hybrid 는 `49ca3150c68d527a275c855ecf814122ff9c533a` 의 `crates/vigilode-hybrid` 및 `research/hybrid_20261020` 의 실행 계약을 사용한다. 동등한 compiler 버전만 맞추고 모델·출력·linear backend·thread·타이머 범위를 바꾸면 원래 비교를 재현한 것이 아니다.

C.2 각 원자료의 권위 있는 위치

노드	저장소 안의 경로	먼저 확인할 파일
초기 두 campaign	`research/benchmark_results_20261019/`	run1 endpoint CSV, run2 CT CSV와 source 설명
엄격 3 자 비교	`research/strict_solver_comparison_20261019/`	`FINAL_REVIEW_KO.md`, final readback, `REPRODUCE.md`
Banded	`research/banded_exact_reuse_20261020/`	`CRITICAL_REVIEW_KO.md`, final run, 회귀 로그
Native coupled	`research/coupled_native_20261020/`	`CRITICAL_REVIEW_KO.md`, `ROOT_VS_TIME_ERROR.csv`
Residual-first	`research/coupled_economy_20261020/`	`ARCHIVE_RECOVERY.md`, `COST.csv`, `AUDIT.json`
중간점검	`research/midcheck_20261020/`	`measure.py`, `RESULTS.csv`, `PAIRS.json`
Hybrid	`research/hybrid_20261020/`	`RECOVERY_REVIEW_KO.md`, 원래 run, `recovery_20261020/`
독립 연구 originals	`research/cumulative_20261020/`	`ORIGINALS_MANIFEST.json`, `original_archives/`, `original_sources/`

PR75 에서 원래 제외됐던 `REFERENCES.json` 은 일반적인 빈 `RECONSTRUCTION.json` 만으로 복원되지 않는다. `reference_recovery/39357596507/REFERENCES.json.gz` 의 압축해제 바이트를 별도 임시 사본에 넣고, 원래 SHA-256 `93d840405fbe3a095346bcadf089b6ee89cf844927be26f9879034ee0fc63091` 을 확인한 뒤 읽기 검산한다. 이 한 파일의 exact 복구를 위해 timing 을 재실행할 필요는 없다. [R75A]

C.3 Google Drive 백업과 접근

네 원본 ZIP 의 Drive 백업은 아래와 같다. 공유 설정을 공개로 변경하지 않았으므로 소유 계정의 접근권한이 필요하다. 원본 checksum 은 17 장의 표와 동봉 manifest 에 있다.

Archive	원본 Drive 링크
Integration / homotopy	Drive 원본
Derivative-free FVM	Drive 원본
Legacy VODE / Diffsol	Drive 원본
Hybrid AD recovery	Drive 원본

대형 환경 자료는 VigilODE Research Backup 폴더의 Rust part00·01·02 및 RUSTCORE ZIP 이다. 복구는 PR78 의 대형 백업 문서에 기록된 순서·checksum 을 따르며, 이 보고서에는 compiler binary 를 다시 첨부하지 않았다.

부록 D. 출처 등록부와 인용 범위

본문의 `[A…]`, `[S…]`, `[R…]` 는 아래 일차 자료를 가리킨다. 외부 `[E…]`, `[B…]` 는 용어·역사·문헌 맥락의 좁은 보조 근거다. 프로젝트의 실측과 새 구성의 검증을 외부 문헌으로 대체하지 않는다. 첨부 문서는 2026 년 10 월 22 일 제공된 사본을 사용하며 파일명과 local hash 는 `qa/INPUT_MANIFEST.json` 에 보존한다.

D.1 현재 첨부와 독립 연구 보고서

[A1] 초기 실행환경 및 첫 벤치마크. `Pasted text(20261022-105426).txt`. 실측 source `ccaeaa50…`. Rust 설치·서명·빌드 조건, 154 endpoint 조건, MF·dense·A07 관측의 근거다. 동봉 읽기 사본: `sources/S01_initial.txt`.

[A2] 정밀 재벤치마크. `Pasted text (2)(20261022-105442).txt`. 실제 빌드 `8a8bd409…`와 후속 `a21d573…`를 구별한다. Old/new 비교, CT 2,700 조건, allocation·Callgrind 및 분석기 결함의 근거다. 동봉: `sources/S02_v2.txt`.

[A3] 엄격 native 3 자 비교의 대화 보고서. `Pasted text (3)(20261022-105511).txt`. PR71 의 source·run·결과·초기값 실패·해석 정정을 기록한다. 동봉: `sources/S03_strict.txt`.

[A4] 미분 없는 FVM 연구의 대화 요약. `Pasted text (4)(20261022-105700).txt`. 뒤의 A6 와 같은 연구를 독자용 설명으로 재구성한 자료다. 별도의 두 실험으로 중복 집계하지 않는다. 동봉: `sources/S09_derivative_summary.txt`.

[A5] 하이브리드 AD 복구 및 비판 분석. `Pasted markdown (5)(20261022-105740).md`. PR77 의 실제 코드·원래 실행·후속 제한된 복구 검산·타이머 범위를 구별한다. 당시의 "추가 push 미수행" 은 이후 R78 출판 사건으로 갱신하되 원문을 바꾸지 않는다. 동봉: `sources/S10_hybrid.md`.

[A6] 미분 없는 causal 격자·packet·adaptive 전체 연구. `Pasted markdown (6)(20261022-105813).md`. 기존 PR 의 재사용 근거, Volterra·reaction ideal·Schur·Bernstein·remap·ALE 의 전제와 실행 범위를 기록한다. 동봉: `sources/S09_derivative_full.md`.

[S05] VigilODE 통합 연구: matrix-free, 단계 병렬 Rosenbrock, homotopy 의 결합. 기존 대화의 `REPORT_KO.md` 전체를 추가로 읽었다. 조사 기준 `c03ca90f…`; Python prototype 13 시험, 30 설정, 큰 projected 예제 및 rank/threading 반례를 포함한다. 동봉: `sources/S05_integration.md`. 원본 ZIP 은 PR78 의 `original_archives/VigilODE_Integration_Research_20261020.zip` 에 있다.

[S12] VigilODE vs VODE/DVODE vs Diffsol (2026-10-19). 기존 대화의 `COMPARISON_REPORT.md`. SciPy 1.17 wrapper 에 대한 초기 관측과 당시의 미정합 비교 상태를 기록한다. 이후 R71 의 원인 확인으로 원본 VODE 에 대한 해석이 수정됐음을 함께 읽어야 한다. 동봉: `sources/S12_legacy_vode.md`.

D.2 고정 커밋의 연구 보고서·구현

[R71] Strict solver comparison. 고정된 최종 비판 분석. A3 와 함께 actual-error 정합, native Netlib comparator, 보존된 초기 bit FAIL 및 raw 저장 한계의 근거다.

[R73] Banded exact-reuse critical review. 고정 보고서. 네 실제 결함·반증된 retry 의심·고정폭 특화·정확 cache·workspace timeout 의 근거다.

[R73R] Banded 고정 증거 복구. 이 스레드의 PR73 후속 복구 메시지와 고정 연구 디렉터리. 복구는 새 integration/timing 을 생성한 사건이 아니다. 본 집필은 회수된 보고서와 스레드 기록을 통합했으며 해당 모든 원자료를 다시 실행하지 않았다.

[R74] Native coupled-stage Rust study. 고정 비판 분석. 모델 scope, 후보의 operational 지위, 19 시험·800 프로세스, root/embedded 승인 분모와 CodeRabbit 제한을 기록한다.

[R75] Residual-first economy. 고정 비판 분석. 33 시험, 동일 결과의 비용 절감, cap64/256 의 부정 결과와 전체 작업량의 근거다.

[R75A] Exact reference archive recovery. 복구 기록. 16,024,519 바이트 참조 파일의 누락 원인과 원래 checksum 에 일치한 복구를 기록한다.

[R76] Same-host midcheck. 고정 보고서. 39 설정·312 측정·30 쌍, 그중 26 개의 기록된 정합 판정을 근거로 한다.

[C76] 실제 중간점검의 오차 승인 코드. 고정 `measure.py`, 결과 집계 구간. 구현은 `max(E) <= 1.1*target` 및 관측 오차비 `<=2` 를 사용한다. 본문 11 장은 이 코드 조건을 PR71 의 엄격 anchor 와 구분한다. 과거 26 쌍을 이번 집필에서 더 엄격한 기준으로 재분류하지 않았다.

[R77] Hybrid structure/local-AD certified trajectory. 실측 소스, 원래 증거, 후속 복구 보고서. A5 는 이 후속 보고서의 첨부 사본이다.

[R78] 누적 아카이브. 고정 인벤토리, 원본 장부, 원격 closeout. 이 통합 편집에서 PR78 metadata 를 재조회해 Draft·미병합·해당 HEAD 를 확인했다.

D.3 외부 공식 자료와 원 논문

[E1] SciPy PR #24934. “BUG: fix VODE Jacobian transposition in dvode_jacobian_thunk (gh-24933).” 공식 변경 기록, merge 2026-03-29, commit `9d88ff1739705d8c9f483eca014d6293d499c260`. 기존 보고서의 wrapper 원인 교정을 확인하는 용도로만 사용했다. 이 변경이 모든 잔존 dense-path 이상을 설명한다고 확대하지 않는다.

[E2] Netlib VODE 원본. `vode.f`. MF21/MF24 interface 와 local tolerance 가 global error 보장은 아니라는 경고를 확인했다. 실제 benchmark comparator 의 bytes 는 각 연구 run 의 source checksum 을 따른다.

[E3] LLNL SUNDIALS usage notes. 공식 usage notes, CVODE 사용 문서. Reinitialization·history 및 dense-output/step 동작에 대한 좁은 확인이다. 강한 AMR 성능 주장을 이 문서가 입증한다고 해석하지 않는다.

[B1] Brown, P. N., Byrne, G. D., and Hindmarsh, A. C. (1989). “VODE: A Variable-Coefficient ODE Solver.” *SIAM Journal on Scientific and Statistical Computing*. DOI: 10.1137/0910062. 원본 VODE 의 방법·계보를 설명하는 서지적 배경이며 이 프로젝트의 새 시간표를 뒷받침하는 자료는 아니다.

[B2] Steihaug, T., and Wolfbrandt, A. (1979). “An Attempt to Avoid Exact Jacobian and Nonlinear Equations in the Numerical Solution of Stiff Differential Equations.” *Mathematics of Computation*. DOI: 10.1090/S0025-5718-1979-0521273-8. 근사 Jacobian 을 허용하는 별도 방법 설계와 exact factor reuse 가 다르다는 문헌적 맥락이다. 본 집필에서 논문의 모든 차수 조건을 재증명하거나 기존 RODAS5P 계수가 임의 W 를 허용한다고 결론내리지 않았다.

Exa 와 SciSpace 는 위 공식 변경 기록·서지사항을 좁게 재확인하는 데 사용했다. 외부 검색을 새로운 연구 설계의 원전이나 역사적 benchmark 의 독립 실행으로 사용하지 않았다.

부록 E. 이번 통합 집필에서 수행한 검증

이 부록은 과거 연구의 재실행과 구분되는 **2026 년 10 월 22 일 편집 검증**을 기록한다.

첫째, 여섯 첨부와 기존 통합·초기 비교 보고서를 전체 읽고 source register 를 만들었다. 읽기 사본과 SHA-256 을 보존했다. 네 독립 ZIP 의 크기·SHA-256·ZIP CRC 를 확인했으며, 대형 V2 원자료 전체나 모든 GitHub 실행을 다시 내려받아 재적분하지 않았다.

둘째, 대표 항등식을 Wolfram 의 정확 기호 연산으로 다시 확인했다. 세 node 반사 격자의 평균·분산 식, band-plus-rank-one Jacobian 과 row sum, 국소 반응의 두 점 항등식·판별식, 동일한 2×2 대각 block 을 갖는 세-stage 하삼각 행렬의 determinant 관계가 모두 영 잔차를 반환했다. 세-stage 유한 대조는 일반 여덟-stage 정리의 독립 형식증명이 아니라, 본문의 block-triangular 유도와 구현 표기의 교차 점검이다. 도구가 형식 변수에 대한 경고를 함께 반환한 사실도 검증 기록에 남겼다.

셋째, 초기·정합·coupled·hybrid 표의 분모·단위·관측 범위를 점검했다. 원문의 per-arm median 비율과 paired-ratio median 을 구분했고, 표의 일부 반올림 때문에 마지막 자리가 다른 값을 새로운 측정으로 만들지 않았다. PR76 의 `1.1*anchor` 는 원전 코드에서 확인한 편집상 주석이며, 원래 측정이나 gate 를 변경한 사건이 아니다.

넷째, Academic Writing Toolkit 의 문단 논리 점검을 핵심 결론의 서술 검사에 사용했다. 도구는 짧은 문단에 대한 편집상 지적 1 개를 반환했으며, 두 BibTeX 레코드의 오프라인 구조 검사에서는 문제를 보고하지 않았다. 온라인 서지 검증이나 과학적 승인으로 해석하지 않았다. 이 도구의 결과는 수학적 정리 검증이나 독립인원 과학 심사를 대신하지 않는다. 최종 문서의 인용 식별자와 source register 연결, 수식·표·페이지 넘침은 별도로 점검한다.

본 통합본의 Markdown 은 편집 가능한 원문이고, PDF 는 같은 내용의 열람본이다. 동봉한 `qa/`에는 편집 검증 기록, `sources/`에는 입력 사본, `SOURCE_REGISTER.json` 에는 출처 연결을 둔다. 이 작업에서 원격 repository 를 수정하거나 새로운 PR 을 열지 않았다.